[TRƯỜNG ĐẠI HỌC / HỌC VIỆN ĐÀO TẠO — PLACEHOLDER]

[KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN HỆ THỐNG THÔNG TIN]

BÁO CÁO BÀI TẬP LỚN HỌC PHẦN CƠ SỞ DỮ LIỆU

ĐỀ TÀI:
HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
(CINEVIET CINEMA CHAIN MANAGEMENT SYSTEM)

Công nghệ: Java 21 LTS • Java Swing (FlatLaf) • MySQL Community Server 8.4 LTS • JDBC • Apache PDFBox 3

Thông tin Học phần	Nội dung Đăng ký (Placeholder)
Giảng viên hướng dẫn	[Họ và tên Giảng viên hướng dẫn — Placeholder]
Lớp học phần / Nhóm	[Mã lớp học phần] — [Nhóm XX — Placeholder]
Sinh viên thực hiện 1	[Họ và tên Sinh viên 1] — MSSV: [Mã SV 01]
Sinh viên thực hiện 2	[Họ và tên Sinh viên 2] — MSSV: [Mã SV 02]
Sinh viên thực hiện 3	[Họ và tên Sinh viên 3] — MSSV: [Mã SV 03]
Hệ quản trị CSDL thực thi	**MySQL Community Server 8.4.0 LTS** (`InnoDB`, `utf8mb4_unicode_ci`)

Hà Nội, Năm học 2026

LỜI MỞ ĐẦU & MA TRẬN ĐÁP ỨNG YÊU CẦU HỌC PHẦN (R1 – R8)

Báo cáo này trình bày trọn vẹn quá trình phân tích, thiết kế cơ sở dữ liệu chuẩn 3NF, cài đặt ràng buộc toàn vẹn & giao dịch đồng thời trên **MySQL Community Server 8.4.0 LTS**, phát triển ứng dụng Desktop **Java 21 LTS + Java Swing (FlatLaf)** kết nối qua **MySQL Connector/J** và kiểm chứng thực nghiệm cho **Hệ thống Quản lý Chuỗi Rạp Chiếu Phim CineViet**. Toàn bộ tài liệu, sơ đồ, mã lệnh SQL, biểu mẫu giao diện và báo cáo xuất PDF đều được đối chiếu nghiêm ngặt theo 8 tiêu chí chấm điểm học phần (tổng cộng 10,0 điểm):

Mã YC	Nội dung Yêu cầu Đánh giá	Điểm	Chương / Mục trong Báo cáo	Trạng thái Kiểm chứng
R1	Phân tích yêu cầu nghiệp vụ và dữ liệu cần lưu trữ	1.0	Chương 1	Đã kiểm chứng trên MySQL 8.4 (PASS)
R2	Thiết kế đầy đủ Sơ đồ Thực thể – Liên kết (ERD Chen & Crow's Foot)	2.5	Chương 2 (Hình 2.1 & 2.2)	Đã kiểm chứng (PASS)
R3	Thiết kế Lược đồ CSDL Quan hệ (13 bảng) và Từ điển dữ liệu	2.0	Chương 3 & Phụ lục A, B	Đã kiểm chứng trên MySQL 8.4 (PASS)
R4	Phân tích Phụ thuộc hàm, tìm khóa và chứng minh chuẩn hóa 3NF	1.0	Chương 4	Đã kiểm chứng (PASS)
R5	Cài đặt 02 màn hình nhập liệu có kiểm tra hợp lệ & chống xung đột	1.0	Chương 5 (Mục 5.2 & 5.3)	Đã kiểm chứng trên MySQL 8.4 (PASS)
R6	Cài đặt 01 màn hình tìm kiếm vận hành đa tiêu chí kèm số ghế trống	0.5	Chương 5 (Mục 5.4)	Đã kiểm chứng trên MySQL 8.4 (PASS)
R7	Cài đặt 02 báo cáo thực tiễn có xem trước (Preview) và xuất PDF	1.0	Chương 5 (Mục 5.5 & 5.6)	Đã kiểm chứng trên MySQL 8.4 (PASS)
R8	Báo cáo học thuật tổng hợp đầy đủ R1–R7 và Phụ lục SQL thực thi	1.0	Toàn bộ Báo cáo & Phụ lục SQL	Đã kiểm chứng (PASS)

CHƯƠNG 1. PHÂN TÍCH YÊU CẦU NGHIỆP VỤ VÀ DỮ LIỆU CẦN LƯU TRỮ (TIÊU CHÍ R1)

(Tài liệu đáp ứng Tiêu chí R1 — 1.0 điểm)

1. Bối cảnh và Bài toán Quản lý Chuỗi Rạp Chiếu Phim

Doanh nghiệp **CineViet Cinema Chain** vận hành một chuỗi rạp chiếu phim hiện đại tại nhiều tỉnh/thành phố lớn (Hà Nội, TP. Hồ Chí Minh, Đà Nẵng, Hải Phòng...). Mỗi chi nhánh rạp quản lý nhiều phòng chiếu tiêu chuẩn khác nhau (`2D`, `3D`, `IMAX`), mỗi phòng chiếu có sơ đồ ghế vật lý cố định phân chia theo hạng ghế (`STD` – Ghế thường, `VIP` – Ghế VIP, `COUPLE` – Ghế đôi).

Hàng ngày, hệ thống phải xử lý đồng thời các quy trình nghiệp vụ cốt lõi:

1. **Quản lý danh mục phim và lập lịch chiếu (Screening Management):** Phân bổ phim vào từng phòng chiếu thuộc từng chi nhánh theo khung giờ cụ thể, đảm bảo thời lượng suất chiếu đủ cho thời lượng phim cộng thời gian dọn phòng/quảng cáo và tuyệt đối không chồng lấn lịch chiếu trong cùng một phòng.

2. **Bán vé và đặt chỗ theo sơ đồ ghế thời gian thực (Ticket Booking):** Phục vụ cả khách hàng thành viên lẫn khách vãng lai tại quầy. Hệ thống phải hiển thị trực quan ghế trống/ghế đã bán, tính đúng đơn giá vé từng ghế (giá cơ sở của suất chiếu + phụ thu hạng ghế tại thời điểm mua), xử lý thanh toán nguyên tử và ngăn chặn tuyệt đối việc một ghế bị bán hai lần cho cùng một suất chiếu ngay cả khi có nhiều nhân viên thu ngân thao tác đồng thời.

3. **Xử lý hủy vé, hoàn tiền và mở bán lại ghế (Ticket Cancellation & Seat Resale):** Khi đơn đặt vé hoặc vé bị hủy hợp lệ, ghế phải được giải phóng ngay để có thể bán lại cho khách hàng khác mà vẫn giữ nguyên lịch sử giao dịch phục vụ kiểm toán.

4. **Tra cứu lịch chiếu và kiểm tra ghế trống (Screening Search):** Cho phép nhân viên và khách hàng tìm kiếm nhanh các suất chiếu theo tên phim, chi nhánh, ngày chiếu, khoảng thời gian và xem ngay số lượng ghế còn trống.

5. **Tổng hợp báo cáo quản trị và tài chính (Business & Financial Reporting):**

• **Báo cáo Doanh thu (Revenue Report):** Do **Kế toán** lập, trình **Giám đốc Kinh doanh chuỗi rạp**, tổng hợp doanh thu thực thu (chỉ tính các vé hợp lệ `VALID` thuộc các đơn đã thanh toán thành công `SUCCESS`), không bị nhân bản dữ liệu khi một đơn có nhiều lần thử thanh toán.

• **Báo cáo Tình hình Bán vé & Lấp đầy (Ticket Sales Report):** Do **Quản lý Chi nhánh** lập, gửi **Ban Điều hành và Quản lý Kinh doanh**, thống kê số vé bán ra, sức chứa phòng, số ghế trống, tỷ lệ lấp đầy (%) và doanh thu theo từng suất chiếu.

2. Phân tích Các Nhóm Người dùng và Phân quyền Nghiệp vụ

Vai trò (Role)	Mã vai trò	Trách nhiệm chính	Màn hình / Báo cáo sử dụng
Thu ngân / Nhân viên bán vé	`CASHIER`	Tra cứu suất chiếu, tư vấn chỗ ngồi, lập đơn đặt vé cho khách thành viên hoặc khách vãng lai, thu tiền và xử lý hủy vé theo quy định.	Màn hình Tra cứu Suất chiếu, Màn hình Đặt vé & Chọn ghế
Quản lý Chi nhánh	`MANAGER`	Lập và điều chỉnh lịch chiếu tại các phòng chiếu của chi nhánh, theo dõi tỷ lệ lấp đầy phòng chiếu và lập Báo cáo Tình hình Bán vé gửi Ban Điều hành & Kinh doanh.	Màn hình Quản lý Suất chiếu, Màn hình Tra cứu, Báo cáo Tình hình Bán vé

Vai trò (Role)	Mã vai trò	Trách nhiệm chính	Màn hình / Báo cáo sử dụng
Kế toán Chuỗi rạp	`ACCOUNTANT`	Kiểm soát giao dịch thanh toán, đối soát doanh thu thực tế theo ngày/kỳ hạn và chi nhánh, lập Báo cáo Doanh thu trình Giám đốc Kinh doanh chuỗi rạp.	Báo cáo Doanh thu Chuỗi rạp, Báo cáo Tình hình Bán vé
Giám đốc Kinh doanh & Ban Điều hành	(Người nhận báo cáo)	Nhận và phê duyệt các báo cáo định kỳ từ Kế toán và Quản lý Chi nhánh để ra quyết định phân bổ suất chiếu, điều chỉnh giá vé và chiến lược khai thác.	Bản in/PDF Báo cáo Doanh thu & Báo cáo Tình hình Bán vé

3. Phân tích Chi tiết Dữ liệu Cần Lưu trữ (13 Thực thể Chuẩn)

Để đảm bảo tính toàn vẹn dữ liệu, không dư thừa và đạt chuẩn **3NF**, hệ thống tổ chức lưu trữ thành **13 tập thực thể / bảng quan hệ** sau đây:

3.1. Nhóm dữ liệu Hạ tầng Rạp & Phòng chiếu

1. **cinema_branch (Chi nhánh rạp):**

• **Ý nghĩa:** Lưu thông tin từng cụm rạp trong chuỗi.

• **Dữ liệu cần lưu:** Mã định danh chi nhánh (`branch_id`), mã nghiệp vụ (`branch_code`, VD: `CN-HN01`), tên chi nhánh (`branch_name`), địa chỉ (`address`), thành phố (`city`), số điện thoại liên hệ (`phone`), trạng thái hoạt động (`active`).

2. **screening_room (Phòng chiếu):**

• **Ý nghĩa:** Lưu các phòng chiếu thuộc từng chi nhánh rạp.

• **Dữ liệu cần lưu:** Mã định danh phòng (`room_id`), mã chi nhánh sở hữu (`branch_id`), tên phòng (`room_name`, VD: `Phòng 01`), loại công nghệ chiếu (`room_type`: `2D`, `3D`, `IMAX`), trạng thái hoạt động (`active`). Lưu ý chuẩn hóa: Tổng số ghế của phòng không lưu cứng để tránh dị thường cập nhật mà được tính tự động từ bảng `seat`.

3. **seat_type (Hạng ghế):**

• **Ý nghĩa:** Danh mục phân loại hạng ghế và mức phụ thu tiêu chuẩn.

• **Dữ liệu cần lưu:** Mã hạng ghế (`seat_type_code`: `STD`, `VIP`, `COUPLE`), tên hiển thị tiếng Việt (`type_name`: Ghế Thường, Ghế VIP, Ghế Đôi), mức phụ thu chuẩn (`surcharge`, VNĐ).

4. **seat (Ghế vật lý):**

• **Ý nghĩa:** Lưu từng vị trí ghế vật lý cố định bên trong một phòng chiếu.

• **Dữ liệu cần lưu:** Mã ghế (`seat_id`), mã phòng chiếu (`room_id`), hàng ghế (`seat_row`: `A`..`H`), số thứ tự ghế trong hàng (`seat_number`: `1`..`12`), mã hạng ghế (`seat_type_code`).

3.2. Nhóm dữ liệu Phim, Thể loại & Lịch chiếu

5. **movie (Phim điện ảnh):**

• **Ý nghĩa:** Lưu thông tin các bộ phim được khai thác tại chuỗi rạp.

• **Dữ liệu cần lưu:** Mã định danh phim (`movie_id`), mã phim (`movie_code`, VD: `MV-001`), tựa đề tiếng Việt (`title`), thời lượng chiếu tính bằng phút (`duration_minutes`), ngày khởi chiếu (`release_date`), phân loại độ tuổi (`age_rating`: `P`, `K`, `T13`, `T16`, `T18`), quốc gia sản xuất (`country`), mô tả tóm tắt (`description`).

6. **genre (Thể loại phim):**

• **Ý nghĩa:** Danh mục các thể loại điện ảnh (Hành động, Tâm lý, Kinh dị, Hoạt hình, Khoa học viễn tưởng...).

• **Dữ liệu cần lưu:** Mã thể loại (`genre_id`), tên thể loại (`genre_name`), mô tả (`description`).

7. **movie_genre (Phân loại Phim – Thể loại):**

• **Ý nghĩa:** Quan hệ nhiều – nhiều (M:N) cho phép một bộ phim thuộc nhiều thể loại và một thể loại có nhiều bộ phim.

• **Dữ liệu cần lưu:** Khóa chính phức hợp (`movie_id`, `genre_id`).

8. **screening (Suất chiếu):**

• **Ý nghĩa:** Một lần trình chiếu cụ thể của một bộ phim tại một phòng chiếu xác định trong một khoảng thời gian.

• **Dữ liệu cần lưu:** Mã suất chiếu (`screening_id`), mã phim (`movie_id`), mã phòng chiếu (`room_id`), thời điểm bắt đầu (`start_time`), thời điểm kết thúc dự kiến bao gồm quảng cáo và vệ sinh phòng (`end_time`), giá vé cơ sở của suất chiếu (`base_price`), trạng thái suất chiếu (`status`: `OPEN`, `CANCELLED`).

3.3. Nhóm dữ liệu Khách hàng, Nhân sự, Đặt vé & Thanh toán

9. **customer (Khách hàng thành viên):**

• **Ý nghĩa:** Lưu thông tin khách hàng đăng ký thành viên để tích lũy lịch sử đặt vé và chăm sóc khách hàng.

• **Dữ liệu cần lưu:** Mã khách hàng (`customer_id`), họ và tên (`full_name`), số điện thoại (`phone`), địa chỉ email (`email`), ngày sinh (`birth_date`), thời điểm đăng ký (`created_at`).

10. **employee (Nhân viên):**

• **Ý nghĩa:** Lưu thông tin nhân sự làm việc tại các chi nhánh (Thu ngân, Quản lý chi nhánh, Kế toán).

• **Dữ liệu cần lưu:** Mã nhân viên (`employee_id`), mã chi nhánh làm việc (`branch_id`), mã nhân sự (`employee_code`), họ và tên (`full_name`), số điện thoại (`phone`), vai trò (`role`: `CASHIER`, `MANAGER`, `ACCOUNTANT`), ngày vào làm (`hired_date`).

11. **booking (Đơn đặt vé):**

• **Ý nghĩa:** Ghi nhận một giao dịch đặt mua vé của khách hàng cho một suất chiếu cụ thể.

• **Dữ liệu cần lưu:** Mã đơn (`booking_id`), mã đơn hiển thị (`booking_code`, VD: `BK-20261101-0001`), mã suất chiếu (`screening_id`), mã khách hàng (`customer_id` — cho phép `NULL` đối với quy trình **Khách vãng lai tại quầy**), mã nhân viên thực hiện (`employee_id`), thời điểm lập đơn (`booked_at`), trạng thái đơn (`booking_status`: `CONFIRMED`, `PENDING`, `CANCELLED`).

12. **ticket (Vé xem phim chi tiết theo ghế):**

• **Ý nghĩa:** Mỗi dòng tương ứng với một ghế cụ thể được bán ra trong một đơn đặt vé của một suất chiếu.

• **Dữ liệu cần lưu:** Mã vé (`ticket_id`), mã đơn đặt vé (`booking_id`), mã suất chiếu (`screening_id`), mã ghế (`seat_id`), đơn giá bán thực tế tại thời điểm xuất vé (`unit_price`), trạng thái vé (`ticket_status`: `VALID`, `CANCELLED`), thời điểm hủy (`cancelled_at`), và cột khóa sinh kiểm soát ghế đang hiệu lực (`active_seat_key`).

13. **payment (Giao dịch Thanh toán):**

• **Ý nghĩa:** Lưu lịch sử các lần thanh toán gắn với một đơn đặt vé (cho phép ghi nhận cả các lần thanh toán thất bại `FAILED` trước khi thanh toán thành công `SUCCESS`, hoặc hoàn tiền `REFUNDED` khi hủy đơn).

• **Dữ liệu cần lưu:** Mã thanh toán (`payment_id`), mã đơn đặt vé (`booking_id`), số tiền thanh toán (`amount`), phương thức thanh toán (`payment_method`: `CASH`, `CARD`, `TRANSFER`, `EWALLET`), trạng thái thanh toán (`payment_status`: `PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`), thời điểm thanh toán (`paid_at`), mã tham chiếu giao dịch (`transaction_ref`), và cột sinh kiểm soát duy nhất 1 thanh toán thành công (`active_success_booking`).

4. Khối lượng Dữ liệu Lưu trữ & Quy mô Khởi tạo (Seed Data Scale)

Để phản ánh chân thực hoạt động của chuỗi rạp và phục vụ kiểm chứng các báo cáo quản trị, cơ sở dữ liệu được khởi tạo tất định với quy mô tối thiểu:

- **04 Chi nhánh rạp** tại Hà Nội, TP. Hồ Chí Minh, Đà Nẵng và Hải Phòng.
- **12 Phòng chiếu** (mỗi chi nhánh 3 phòng gồm `2D`, `3D`, `IMAX`), tổng cộng **960 ghế vật lý** (mỗi phòng 8 hàng A–H × 10 ghế = 80 ghế phân bổ `STD`, `VIP`, `COUPLE`).
- **08 Thể loại phim** và **20 Bộ phim** điện ảnh có đầy đủ phân loại độ tuổi, thời lượng thực tế.
- **12 Nhân viên** (thu ngân, quản lý chi nhánh, kế toán) và **45 Khách hàng thành viên** (cùng quy trình khách vãng lai).
- **160 Suất chiếu** phân bổ hợp lệ không trùng lịch trên 12 phòng chiếu.
- **160 Đơn đặt vé** với **373 Vé xem phim** (bao gồm 23 ghế hủy được bán lại hợp lệ) và **175 Giao dịch thanh toán** bao phủ toàn bộ các kịch bản: thanh toán thành công (`SUCCESS`), đang chờ (`PENDING`), thất bại (`FAILED`), hoàn tiền khi hủy (`REFUNDED`), đơn có thử lại thanh toán sau khi lỗi thẻ, và ghế bị hủy được bán lại thành công cho khách mới.

Quy tắc Nghiệp vụ & Cơ chế Đảm bảo Toàn vẹn (BR-01 .. BR-16)

(Tài liệu đáp ứng Tiêu chí R1 & R3 — Business Rules Specification)

1. Bảng Ánh xạ Quy tắc Nghiệp vụ (Business Rules Traceability)

Mỗi quy tắc nghiệp vụ dưới đây được định danh duy nhất (`BR-01` .. `BR-16`) và được bảo vệ đồng thời ở **Tầng Cơ sở Dữ liệu (DDL / Trigger / Index)** và **Tầng Dịch vụ Ứng dụng (Java Service & Transaction)**:

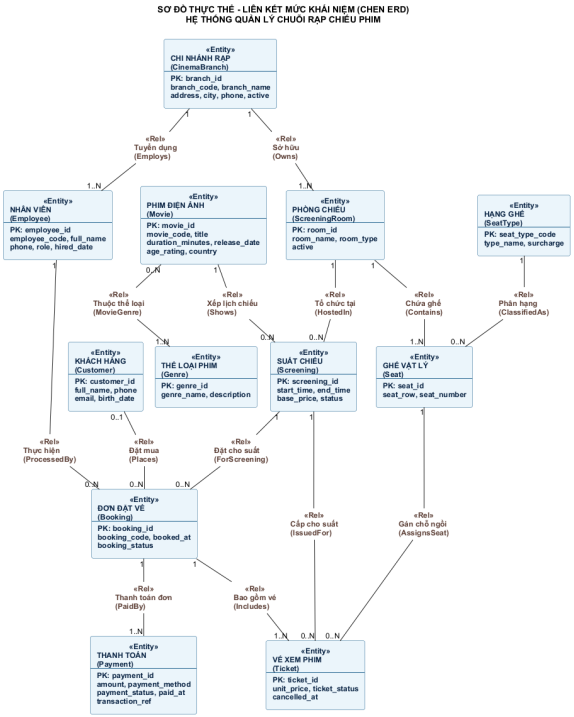
Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-01	Một chi nhánh rạp (`cinema_branch`) sở hữu nhiều phòng chiếu (`screening_room`). Tên phòng chiếu là duy nhất trong phạm vi cùng một chi nhánh.	Khóa ngoại `fk_room_branch` (`branch_id NOT NULL`); Ràng buộc duy nhất `uq_room_branch_name (branch_id, room_name)`.	Kiểm tra chi nhánh tồn tại và đang hoạt động trước khi lọc hoặc gán phòng chiếu.
BR-02	Mỗi phòng chiếu chứa nhiều ghế vật lý (`seat`). Tọa độ `(room_id, seat_row, seat_number)` là duy nhất trong mỗi phòng, mỗi ghế thuộc một hạng ghế (`seat_type`) hợp lệ.	Khóa ngoại `fk_seat_room`, `fk_seat_type`; Ràng buộc duy nhất `uq_seat_position (room_id, seat_row, seat_number)`.	Hiển thị sơ đồ ghế trực tiếp từ danh sách ghế của phòng chiếu thuộc suất chiếu được chọn.
BR-03	Một bộ phim (`movie`) có thể thuộc nhiều thể loại (`genre`) và một thể loại có thể gắn với nhiều bộ phim; thời lượng phim phải từ 1 đến 400 phút.	Bảng trung gian `movie_genre` có khóa chính phức hợp `PRIMARY KEY (movie_id, genre_id)`; `CHECK (duration_minutes BETWEEN 1 AND 400)`.	Kiểm tra thời lượng phim khi tính toán thời gian kết thúc tối thiểu của suất chiếu.
BR-04	Một bộ phim có thể được xếp nhiều suất chiếu (`screening`), nhưng mỗi suất chiếu chỉ thuộc duy nhất một phòng chiếu và một bộ phim xác định; giá cơ sở phải dương (`base_price > 0`).	Khóa ngoại `fk_screening_movie`, `fk_screening_room` (`NOT NULL`); `CHECK (base_price > 0)`.	`ScreeningService.validateScreeningInput()` kiểm tra phim, phòng, ngày giờ và giá cơ sở hợp lệ.

Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-05	Thời gian kết thúc suất chiếu (`end_time`) phải lớn hơn thời gian bắt đầu (`start_time`) tối thiểu bằng thời lượng bộ phim (`movie.duration_minutes`).	`CHECK (end_time > start_time)` + Trigger `trg_screening_validate_ins` và `trg_screening_validate_upd` kiểm tra `TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) >= duration_minutes`.	`ScreeningService` kiểm tra `endTime >= startTime.plusMinutes(movie.durationMinutes())` và gợi ý tự động thời gian kết thúc (+15 phút dọn phòng).
BR-06	**Chống trùng lịch chiếu trong cùng một phòng chiếu (kể cả khi thao tác đồng thời):** Hai suất chiếu có trạng thái `OPEN` trong cùng một `room_id` không được giao nhau về khoảng thời gian: không tồn tại suất chiếu khác có `existing.start_time < new.end_time AND existing.end_time > new.start_time`.	Trigger `trg_screening_validate_ins` & `trg_screening_validate_upd` truy vấn bảng `screening` và phát lỗi `SIGNAL SQLSTATE '45000'` nếu phát hiện chồng lấn.	Trong `ScreeningService`, mở giao dịch và thực hiện `SELECT room_id FROM screening_room WHERE room_id = ? FOR UPDATE` để tuần tự hóa mọi yêu cầu lập lịch trên cùng phòng chiếu trước khi kiểm tra trùng lịch và ghi dữ liệu.
BR-07	Mỗi đơn đặt vé (`booking`) phải gắn với một khách hàng thành viên hợp lệ (`customer_id` tham chiếu `customer`) **hoặc** thuộc quy trình **Khách vãng lai tại quầy** (`customer_id IS NULL`) được chọn tường minh trên giao diện; và gắn với nhân viên thực hiện (`employee_id`) cùng suất chiếu (`screening_id`).	Khóa ngoại `fk_booking_customer` (cho phép `NULL`), `fk_booking_employee` (`NOT NULL`), `fk_booking_screening` (`NOT NULL`).	Giao diện Đặt vé có tùy chọn rõ ràng "Khách vãng lai (Mua trực tiếp tại quầy)" hoặc chọn khách hàng thành viên từ danh sách.
BR-08	Một đơn đặt vé (`booking`) có thể bao gồm một hoặc nhiều vé (`ticket`), tất cả các vé trong cùng một đơn phải thuộc đúng suất chiếu `booking.screening_id`.	Trigger `trg_ticket_validate_ins` & `trg_ticket_validate_upd` kiểm tra `NEW.screening_id = booking.screening_id` và suất chiếu đang ở trạng thái `OPEN`.	`BookingService` tạo `booking` và toàn bộ danh sách `ticket` trong cùng một giao dịch nguyên tử với cùng `screening_id`.
BR-09	**Ghế trên vé phải thuộc đúng phòng chiếu của suất chiếu (kể cả khi cập nhật suất chiếu):** Mỗi vé (`ticket`) tham chiếu tới một `seat_id` mà `seat.room_id` bắt buộc phải bằng `screening.room_id`; đồng thời cấm đổi `room_id` của suất chiếu đã phát sinh vé.	Trigger `trg_ticket_validate_ins/upd` đối chiếu `seat.room_id = screening.room_id`, và Trigger `trg_screening_validate_upd` từ chối cập nhật `NEW.room_id <> OLD.room_id` nếu `EXISTS (SELECT 1 FROM ticket WHERE screening_id = OLD.screening_id)`. (Thực thi qua Trigger thay vì thêm `room_id` vào `ticket` để giữ lược đồ đạt chuẩn 3NF, không sinh phụ thuộc bắc cầu).	`BookingService` kiểm tra danh sách ghế thuộc phòng của suất chiếu trước khi lưu; `ScreeningService.updateScreening()` khóa suất chiếu và chặn đổi phòng khi đã có vé.

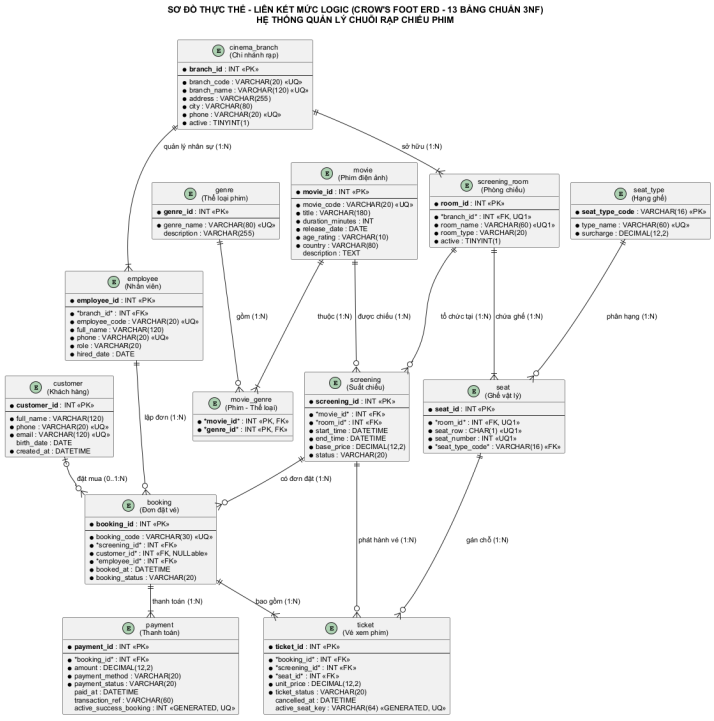
Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-10	**Không bán trùng ghế cho cùng một suất chiếu (kể cả khi đặt vé đồng thời):** Cùng một `seat_id` không bao giờ được có quá một vé ở trạng thái `VALID` trong cùng một `screening_id`.	Cột sinh lưu trữ `active_seat_key VARCHAR(64) GENERATED ALWAYS AS (CASE WHEN ticket_status = 'VALID' THEN CONCAT(screening_id, '-', seat_id) ELSE NULL END) STORED` kèm ràng buộc `UNIQUE KEY uq_ticket_active_seat (active_seat_key)` và Trigger kiểm tra.	`BookingService` khóa dòng suất chiếu `SELECT screening_id, room_id, base_price, status FROM screening WHERE screening_id = ? FOR UPDATE`, kiểm tra các ghế chưa có vé `VALID`, sau đó mới chèn vé.
BR-11	**Tính nhất quán khi hủy vé và bán lại ghế (Cancellation & Seat Resale):** Khi hủy đơn/vé, `ticket_status` chuyển thành `'CANCELLED'`, `cancelled_at` được ghi nhận; cột sinh `active_seat_key` tự động trở thành `NULL` (trong SQL chuẩn, nhiều giá trị `NULL` không vi phạm `UNIQUE`), cho phép ghế đó được đặt lại trong đơn mới mà không cần xóa lịch sử vé đã hủy.	Cột sinh `active_seat_key` + Trigger `trg_booking_status_sync` đồng bộ trạng thái vé và thanh toán.	`BookingService.cancelBooking(bookingId)` cập nhật trong một giao dịch: `booking_status = 'CANCELLED'`, toàn bộ `ticket_status = 'CANCELLED'`, và nếu thanh toán đã `SUCCESS` thì chuyển sang `'REFUNDED'`.
BR-12	**Bảo toàn giá vé lịch sử (Historical Ticket Pricing):** Đơn giá vé `ticket.unit_price` được chốt tại thời điểm bán bằng `screening.base_price + seat_type.surcharge` và lưu trực tiếp vào bảng `ticket`. Mọi thay đổi về sau đối với `screening.base_price` hoặc `seat_type.surcharge` không làm thay đổi doanh thu lịch sử của các vé đã phát hành.	Cột `ticket.unit_price DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0)` lưu giá trị thực tế lúc giao dịch.	`BookingService` tính `unitPrice = screening.basePrice().add(seat.surcharge())` ngay trong giao dịch đặt vé và ghi cố định vào `ticket.unit_price`.
BR-13	**Mỗi đơn đặt vé có tối đa 01 giao dịch thanh toán thành công (SUCCESS) và số tiền thanh toán phải khớp tổng tiền vé hợp lệ:** Một đơn có thể có các lần thanh toán thất bại (`FAILED`) trước đó, nhưng chỉ được có duy nhất một bản ghi `payment_status = 'SUCCESS'`.	Cột sinh `active_success_booking INT GENERATED ALWAYS AS (CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END) STORED` kèm `UNIQUE KEY uq_payment_one_success (active_success_booking)` + Trigger kiểm tra `amount` khớp tổng `unit_price` của các vé `VALID`.	`BookingService` đảm bảo tạo và cập nhật thanh toán nhất quán với tổng tiền vé của đơn.

Mã QT	Phát biểu Quy tắc Nghiệp vụ (Tiếng Việt)	Cơ chế Cưỡng chế tại CSDL (Database Level)	Cơ chế Cưỡng chế tại Ứng dụng (Service / Transaction)
BR-14	**Quy tắc Ghi nhận Doanh thu Chuẩn (Revenue Recognition Rule):** Chỉ những vé có `ticket_status = 'VALID'` thuộc đơn đặt vé `booking_status = 'CONFIRMED'` và có giao dịch thanh toán ở trạng thái `payment_status = 'SUCCESS'` mới được ghi nhận vào doanh thu. Các giao dịch ở trạng thái `PENDING`, `FAILED` hoặc đã hoàn tiền `REFUNDED` (do hủy vé) tuyệt đối bị loại trừ khỏi doanh thu.	View `vw_verified_booking_revenue` và các truy vấn báo cáo lọc chặt chẽ điều kiện `b.booking_status = 'CONFIRMED' AND t.ticket_status = 'VALID' AND p.payment_status = 'SUCCESS'`.	Cả `RevenueReportRepository` và `TicketSalesReportRepository` đều áp dụng cùng một định nghĩa doanh thu và được kiểm chứng chéo bằng SQL độc lập trong `ReportAccuracyIT`.
BR-15	**Chống nhân bản doanh thu do kết nối bảng (Anti-Fanout Join Rule):** Khi một đơn đặt vé có nhiều bản ghi trong bảng `payment` (ví dụ: 1 lần `FAILED` do lỗi thẻ, sau đó 1 lần `SUCCESS`), truy vấn báo cáo tuyệt đối không được nhân đôi số vé hoặc tổng tiền vé.	Ràng buộc `uq_payment_one_success` đảm bảo tối đa 1 dòng `payment` có `payment_status = 'SUCCESS'` cho mỗi `booking_id`; đồng thời truy vấn chỉ join với bản ghi `payment` có `payment_status = 'SUCCESS'`.	Được kiểm chứng bằng dữ liệu mẫu có đơn nhiều lần thanh toán (`FAILED` + `SUCCESS`) và bài test `ReportAccuracyIT`.
BR-16	**Trách nhiệm lập và nhận báo cáo quản trị:** - **Báo cáo Doanh thu:** Người lập là **Kế toán chuỗi rạp**, người nhận là **Giám đốc Kinh doanh chuỗi rạp**. - **Báo cáo Tình hình Bán vé:** Người lập là **Quản lý Chi nhánh**, người nhận là **Ban Điều hành và Quản lý Kinh doanh**.	—	Hiển thị rõ thông tin Người lập, Người nhận, Thời điểm kết xuất, Chữ ký xác nhận trên cả giao diện Xem trước (Preview) và tệp PDF xuất ra.

CHƯƠNG 2. THIẾT KẾ SƠ ĐỒ THỰC THỂ – LIÊN KẾT ERD (TIÊU CHÍ R2)



Hình 2.1. Sơ đồ Thực thể – Liên kết mức Khái niệm theo ký hiệu Chen (Conceptual ERD).



Hình 2.2. Sơ đồ Thực thể – Liên kết mức Logic theo ký hiệu Crow's Foot (Logical ERD — 13 bảng).

(Tài liệu đáp ứng Tiêu chí R2 — 2.5 điểm)

1. Tổng quan Sơ đồ Thực thể – Liên kết (ERD)

Mô hình Thực thể – Liên kết của **Hệ thống Quản lý Chuỗi Rạp Chiếu Phim** được xây dựng gồm **12 tập thực thể chính** và **01 quan hệ kết hợp nhiều – nhiều (movie_genre)**, tương ứng với **13 bảng quan hệ**

chuẩn 3NF khi ánh xạ sang mô hình quan hệ.

Hệ thống cung cấp đồng thời:

1. **Sơ đồ ERD mức khái niệm (Chen Notation):** Mã nguồn tại `docs/erd/erd-chen.puml`, ảnh kết xuất tại `docs/erd/erd-chen.png` và `docs/erd/erd-chen.svg`.
2. **Sơ đồ ERD mức logic (Crow's Foot Notation):** Mã nguồn tại `docs/erd/erd-logical.puml`, ảnh kết xuất tại `docs/erd/erd-logical.png` và `docs/erd/erd-logical.svg`.
3. **Sơ đồ Mermaid ER trực quan:** Hiển thị trực tiếp dưới đây.

2. Thuyết minh Chi tiết Từng Tập Thực thể (Entities)

STT	Tên Thực thể (Code / SQL)	Tên Tiếng Việt	Khóa chính (PK)	Khóa dự tuyển / Duy nhất (UQ)	Giải thích Ý nghĩa Nghiệp vụ
1	`CinemaBranch` (`cinema_branch`)	Chi nhánh rạp	`branch_id`	`branch_code`, `branch_name`, `phone`	Đại diện cho một cụm rạp chiếu phim thuộc chuỗi rạp đặt tại một địa điểm/thành phố cụ thể.
2	`ScreeningRoom` (`screening_room`)	Phòng chiếu	`room_id`	`(branch_id, room_name)`	Không gian chiếu phim vật lý thuộc một chi nhánh rạp, phân loại theo công nghệ `2D`, `3D`, `IMAX`.
3	`SeatType` (`seat_type`)	Hạng ghế	`seat_type_code`	`type_name`	Danh mục phân hạng ghế (`STD` – Ghế Thường, `VIP` – Ghế VIP, `COUPLE` – Ghế Đôi) và phụ thu chuẩn (`surcharge`).
4	`Seat` (`seat`)	Ghế vật lý	`seat_id`	`(room_id, seat_row, seat_number)`	Vị trí chỗ ngồi cố định trong một phòng chiếu, xác định bởi hàng ghế (`A`..`H`) và số thứ tự ghế.
5	`Movie` (`movie`)	Phim điện ảnh	`movie_id`	`movie_code`	Tác phẩm điện ảnh được đưa vào khai thác thương mại với thời lượng (`duration_minutes`) và độ tuổi giới hạn (`age_rating`).
6	`Genre` (`genre`)	Thể loại phim	`genre_id`	`genre_name`	Danh mục thể loại điện ảnh phục vụ phân loại và tìm kiếm phim.
7	`MovieGenre` (`movie_genre`)	Phim – Thể loại	`(movie_id, genre_id)`	—	Thực thể kết hợp giải quyết mối quan hệ M:N giữa Phim và Thể loại.

STT	Tên Thực thể (Code / SQL)	Tên Tiếng Việt	Khóa chính (PK)	Khóa dự tuyển / Duy nhất (UQ)	Giải thích Ý nghĩa Nghiệp vụ
8	`Screening` (`screening`)	Suất chiếu	`screening_id`	—	Một lịch trình chiếu cụ thể của một bộ phim tại một phòng chiếu trong khoảng thời gian `[start_time, end_time)` với mức giá cơ sở `base_price`.
9	`Customer` (`customer`)	Khách hàng	`customer_id`	`phone`, `email`	Khách hàng đăng ký thành viên của chuỗi rạp để lưu lịch sử mua vé và nhận thông tin.
10	`Employee` (`employee`)	Nhân viên	`employee_id`	`employee_code`, `phone`	Nhân sự làm việc tại chi nhánh rạp gồm Thu ngân (`CASHIER`), Quản lý chi nhánh (`MANAGER`) và Kế toán (`ACCOUNTANT`).
11	`Booking` (`booking`)	Đơn đặt vé	`booking_id`	`booking_code`	Phiên giao dịch đặt mua một hoặc nhiều vé cho một suất chiếu xác định (do khách thành viên hoặc khách vãng lai thực hiện).
12	`Ticket` (`ticket`)	Vé xem phim	`ticket_id`	`active_seat_key` (cho các vé VALID)	Tấm vé cụ thể gán một chỗ ngồi (`seat_id`) trong một suất chiếu (`screening_id`) thuộc một đơn đặt vé (`booking_id`) với đơn giá lịch sử `unit_price`.
13	`Payment` (`payment`)	Giao dịch thanh toán	`payment_id`	`active_success_booking` (cho thanh toán SUCCESS)	Bản ghi thanh toán cho đơn đặt vé, lưu số tiền, phương thức và trạng thái (`PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`).

3. Thuyết minh Chi tiết Các Mối Quan hệ và Bản số (Relationships & Cardinalities)

1. **cinema_branch (1) — Sở hữu — (1..N) screening_room:**

• Một chi nhánh rạp phải có ít nhất một phòng chiếu và có thể sở hữu nhiều phòng chiếu. Mỗi phòng chiếu thuộc về duy nhất một chi nhánh (`screening_room.branch_id NOT NULL`).

2. **cinema_branch (1) — Tuyển dụng — (1..N) employee:**

• Một chi nhánh quản lý nhiều nhân viên; mỗi nhân viên biên chế tại một chi nhánh xác định (`employee.branch_id NOT NULL`).

3. **screening_room (1) — Chứa ghế — (1..N) seat:**

• Mỗi phòng chiếu có nhiều ghế vật lý được lắp đặt cố định; mỗi ghế vật lý thuộc duy nhất một phòng chiếu (`seat.room_id NOT NULL`).

4. **seat_type (1) — Phân hạng — (0..N) seat:**

• Một hạng ghế (`STD`, `VIP`, `COUPLE`) áp dụng cho nhiều ghế; mỗi ghế vật lý có đúng một hạng ghế (`seat.seat_type_code NOT NULL`).

5. **movie (0..N) — Thuộc thể loại (movie_genre) — (1..N) genre:**

• Một bộ phim có ít nhất một hoặc nhiều thể loại; một thể loại có thể gắn cho nhiều bộ phim. Biểu diễn qua bảng `movie_genre(movie_id, genre_id)`.

6. **movie (1) — Được xếp lịch — (0..N) screening:**

• Một bộ phim có thể có không hoặc nhiều suất chiếu; mỗi suất chiếu trình chiếu đúng một bộ phim (`screening.movie_id NOT NULL`).

7. **screening_room (1) — Tổ chức tại — (0..N) screening:**

• Một phòng chiếu tổ chức nhiều suất chiếu theo các khung giờ không giao nhau; mỗi suất chiếu diễn ra tại duy nhất một phòng chiếu (`screening.room_id NOT NULL`).

8. **customer (0..1) — Đặt mua — (0..N) booking:**

• Một khách hàng thành viên có thể đặt nhiều đơn vé. Ngược lại, một đơn đặt vé có thể thuộc về một khách hàng thành viên (`customer_id` khác `NULL`) hoặc là **Khách vãng lai mua tại quầy** (`customer_id IS NULL`), do đó bản số phía `customer` là `0..1`.

9. **employee (1) — Thực hiện — (0..N) booking:**

• Một nhân viên thu ngân lập nhiều đơn đặt vé; mỗi đơn đặt vé được ghi nhận bởi đúng một nhân viên phụ trách (`booking.employee_id NOT NULL`).

10. **screening (1) — Có đơn đặt — (0..N) booking:**

• Một suất chiếu có thể có nhiều đơn đặt vé; mỗi đơn đặt vé đặt chỗ cho một suất chiếu xác định (`booking.screening_id NOT NULL`).

11. **booking (1) — Bao gồm — (1..N) ticket:**

• Một đơn đặt vé bao gồm từ một đến nhiều vé xem phim (mỗi vé ứng với một ghế); mỗi vé thuộc về duy nhất một đơn đặt vé (`ticket.booking_id NOT NULL`).

12. **screening (1) & seat (1) — Phát hành vé & Gán chỗ ngồi — (0..N) ticket:**

• Mỗi vé tham chiếu tới suất chiếu (`ticket.screening_id`) và ghế vật lý (`ticket.seat_id`). Ràng buộc duy nhất trên khóa sinh `active_seat_key` đảm bảo tại mọi thời điểm chỉ có tối đa **01 vé hợp lệ (VALID)** cho mỗi cặp `(screening_id, seat_id)`, trong khi các vé đã hủy (`CANCELLED`) vẫn được lưu lịch sử.

13. **booking (1) — Thanh toán — (1..N) payment:**

• Một đơn đặt vé có thể có nhiều bản ghi giao dịch thanh toán (ví dụ: lần 1 quẹt thẻ `FAILED`, lần 2 chuyển khoản `SUCCESS`), nhưng ràng buộc duy nhất trên `active_success_booking` đảm bảo mỗi đơn chỉ có tối đa **01 giao dịch SUCCESS**.

CHƯƠNG 3. LƯỢC ĐỒ CƠ SỞ DỮ LIỆU QUAN HỆ VÀ TỪ ĐIỂN DỮ LIỆU TRÊN MYSQL 8.4 (TIÊU CHÍ R3)

(Tài liệu đáp ứng Tiêu chí R3 — 2.0 điểm)

1. Tổng hợp Danh mục 13 Lược đồ Quan hệ

Quy ước ký hiệu trong biểu diễn lược đồ quan hệ:

- **Gạch chân nét liền (PK)**: Thuộc tính thuộc Khóa chính (Primary Key).
- Chữ nghiêng kèm dấu `#` (`FK`): Thuộc tính đóng vai trò Khóa ngoại (Foreign Key).
- `UQ`: Thuộc tính hoặc bộ thuộc tính có ràng buộc Duy nhất (Unique Key / Alternate Key).

1. **cinema_branch** (`branch_id`, `branch_code` [UQ], `branch_name` [UQ], `address`, `city`, `phone` [UQ], `active`)
2. **screening_room** (`room_id`, `#branch_id`, `room_name`, `room_type`, `active`) — với `UQ(branch_id, room_name)`
3. **seat_type** (`seat_type_code`, `type_name` [UQ], `surcharge`)
4. **seat** (`seat_id`, `#room_id`, `seat_row`, `seat_number`, `#seat_type_code`) — với `UQ(room_id, seat_row, seat_number)`
5. **movie** (`movie_id`, `movie_code` [UQ], `title`, `duration_minutes`, `release_date`, `age_rating`, `country`, `description`)
6. **genre** (`genre_id`, `genre_name` [UQ], `description`)
7. **movie_genre** (`#movie_id`, `#genre_id`)
8. **screening** (`screening_id`, `#movie_id`, `#room_id`, `start_time`, `end_time`, `base_price`, `status`)
9. **customer** (`customer_id`, `full_name`, `phone` [UQ], `email` [UQ], `birth_date`, `created_at`)
10. **employee** (`employee_id`, `#branch_id`, `employee_code` [UQ], `full_name`, `phone` [UQ], `role`, `hired_date`)
11. **booking** (`booking_id`, `booking_code` [UQ], `#screening_id`, `#customer_id` [NULLable], `#employee_id`, `booked_at`, `booking_status`)
12. **ticket** (`ticket_id`, `#booking_id`, `#screening_id`, `#seat_id`, `unit_price`, `ticket_status`, `cancelled_at`, `active_seat_key` [GENERATED, UQ])
13. **payment** (`payment_id`, `#booking_id`, `amount`, `payment_method`, `payment_status`, `paid_at`, `transaction_ref`, `active_success_booking` [GENERATED, UQ])

2. Đặc tả Chi tiết Từng Bảng Quan hệ (Data Dictionary)

Bảng 1: `cinema_branch` — Chi nhánh Rạp chiếu phim

Lưu trữ danh sách các cụm rạp thuộc chuỗi trên toàn quốc.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
`branch_id`	`INT AUTO_INCREMENT`	`NOT NULL`	Auto	`PRIMARY KEY`	Mã định danh tự tăng của chi nhánh rạp.
`branch_code`	`VARCHAR(20)`	`NOT NULL`	—	`UNIQUE (uq_branch_code)`	Mã nghiệp vụ của chi nhánh (VD: `CN-HN01`, `CN-HCM01`).

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
branch_name	VARCHAR(120)	NOT NULL	—	UNIQUE (uq_branch_name)	Tên giao dịch chính thức của chi nhánh rạp.
address	VARCHAR(255)	NOT NULL	—	—	Địa chỉ chi tiết của cụm rạp.
city	VARCHAR(80)	NOT NULL	—	—	Tỉnh / Thành phố nơi đặt chi nhánh.
phone	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_branch_phone)	Số điện thoại đường dây nóng của chi nhánh.
active	TINYINT(1)	NOT NULL	1	CHECK (active IN (0, 1))	Trạng thái hoạt động (1: Đang hoạt động, 0: Tạm ngưng).

Bảng 2: screening_room — Phòng chiếu phim

Lưu thông tin các phòng chiếu thuộc từng chi nhánh rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
room_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của phòng chiếu.
branch_id	INT	NOT NULL	—	FK -> cinema_branch(branch_id), UQ(branch_id, room_name)	Mã chi nhánh rạp sở hữu phòng chiếu này.
room_name	VARCHAR(60)	NOT NULL	—	UQ(branch_id, room_name)	Tên phòng chiếu trong chi nhánh (VD: Phòng 01 - 2D).
room_type	VARCHAR(20)	NOT NULL	'2D'	CHECK (room_type IN ('2D', '3D', 'IMAX'))	Công nghệ phòng chiếu (2D, 3D, IMAX).
active	TINYINT(1)	NOT NULL	1	CHECK (active IN (0, 1))	Trạng thái vận hành của phòng chiếu.

Bảng 3: seat_type — Danh mục Hạng ghế

Lưu cấu hình các loại ghế và mức phụ thu cộng thêm vào giá vé cơ sở của suất chiếu.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
seat_type_code	VARCHAR(16)	NOT NULL	—	PRIMARY KEY, CHECK (seat_type_code IN ('STD', 'VIP', 'COUPLE'))	Mã hạng ghế (STD: Thường, VIP: VIP, COUPLE: Ghế đôi).
type_name	VARCHAR(60)	NOT NULL	—	UNIQUE (uq_seat_type_name)	Tên hiển thị tiếng Việt của hạng ghế.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
surcharge	DECIMAL(12,2)	NOT NULL	0.00	CHECK (surcharge >= 0)	Phụ thu hạng ghế (VNĐ) cộng vào base_price của suất chiếu.

Bảng 4: seat — Ghế vật lý trong Phòng chiếu

Lưu tọa độ từng ghế vật lý thuộc một phòng chiếu xác định.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
seat_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của ghế vật lý.
room_id	INT	NOT NULL	—	FK -> screening_room(room_id), UQ(room_id, seat_row, seat_number)	Mã phòng chiếu chứa ghế này.
seat_row	CHAR(1)	NOT NULL	—	UQ(room_id, seat_row, seat_number), CHECK (seat_row BETWEEN 'A' AND 'Z')	Ký hiệu hàng ghế theo bảng chữ cái (A, B, ..., H).
seat_number	INT	NOT NULL	—	UQ(room_id, seat_row, seat_number), CHECK (seat_number BETWEEN 1 AND 50)	Số thứ tự ghế trong hàng (1, 2, ..., 10).
seat_type_code	VARCHAR(16)	NOT NULL	'STD'	FK -> seat_type(seat_type_code)	Mã hạng ghế áp dụng cho vị trí ghế này.

Bảng 5: movie — Phim điện ảnh

Lưu danh mục phim được chiếu tại chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
movie_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của bộ phim.
movie_code	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_movie_code)	Mã nghiệp vụ của bộ phim (VD: MV-001).
title	VARCHAR(180)	NOT NULL	—	—	Tựa đề chính thức của bộ phim (tiếng Việt).
duration_minutes	INT	NOT NULL	—	CHECK (duration_minutes BETWEEN 1 AND 400)	Thời lượng chiếu thực tế của bộ phim (tính bằng phút).
release_date	DATE	NOT NULL	—	—	Ngày khởi chiếu chính thức.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
age_rating	VARCHAR(10)	NOT NULL	'P'	CHECK (age_rating IN ('P', 'K', 'T13', 'T16', 'T18'))	Phân loại độ tuổi theo quy định Cục Điện ảnh (P, K, T13, T16, T18).
country	VARCHAR(80)	NOT NULL	—	—	Quốc gia sản xuất phim.
description	TEXT	NULL	NULL	—	Tóm tắt nội dung phim.

Bảng 6: genre — Thể loại Phim

Lưu danh mục thể loại điện ảnh.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
genre_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh thể loại phim.
genre_name	VARCHAR(80)	NOT NULL	—	UNIQUE (uq_genre_name)	Tên thể loại phim (Hành động, Tâm lý, Kinh dị...).
description	VARCHAR(255)	NULL	NULL	—	Mô tả chi tiết thể loại.

Bảng 7: movie_genre — Liên kết Phim và Thể loại

Bảng trung gian biểu diễn quan hệ nhiều – nhiều (M:N) giữa movie và genre.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
movie_id	INT	NOT NULL	—	PK(movie_id, genre_id), FK -> movie(movie_id)	Mã bộ phim.
genre_id	INT	NOT NULL	—	PK(movie_id, genre_id), FK -> genre(genre_id)	Mã thể loại gắn cho bộ phim.

Bảng 8: screening — Suất chiếu (Lịch chiếu)

Lưu lịch trình chiếu một bộ phim tại một phòng chiếu xác định.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK /UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
screening_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh suất chiếu.
movie_id	INT	NOT NULL	—	FK -> movie(movie_id)	Mã bộ phim được trình chiếu.
room_id	INT	NOT NULL	—	FK -> screening_room(room_id)	Mã phòng chiếu tổ chức suất chiếu.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
start_time	DATETIME	NOT NULL	—	INDEX idx_screening_room_time (room_id, start_time, end_time)	Thời điểm bắt đầu suất chiếu.
end_time	DATETIME	NOT NULL	—	CHECK (end_time > start_time) + Trigger kiểm tra ≥ start_time + duration_minutes và không chồng lấn lịch phòng	Thời điểm kết thúc suất chiếu (đã gồm thời lượng phim và thời gian nghỉ/vệ sinh phòng).
base_price	DECIMAL(12,2)	NOT NULL	—	CHECK (base_price > 0)	Giá vé cơ sở của suất chiếu (VNĐ) áp dụng cho hạng ghế chuẩn STD.
status	VARCHAR(20)	NOT NULL	'OPEN'	CHECK (status IN ('OPEN', 'CANCELLED'))	Trạng thái suất chiếu (OPEN: Đang mở bán, CANCELLED: Đã hủy suất).

Bảng 9: customer — Khách hàng thành viên

Lưu thông tin khách hàng đăng ký thành viên tại chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
customer_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh khách hàng thành viên.
full_name	VARCHAR(120)	NOT NULL	—	—	Họ và tên đầy đủ của khách hàng.
phone	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_customer_phone)	Số điện thoại liên hệ và định danh thành viên tại quầy.
email	VARCHAR(120)	NOT NULL	—	UNIQUE (uq_customer_email)	Địa chỉ thư điện tử nhận vé của khách hàng.
birth_date	DATE	NULL	NULL	—	Ngày sinh khách hàng (phục vụ kiểm tra độ tuổi phim).
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	—	Thời điểm đăng ký tài khoản thành viên.

Bảng 10: employee — Nhân viên Rạp

Lưu thông tin nhân viên làm việc tại các chi nhánh của chuỗi rạp.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
employee_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của nhân viên.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
branch_id	INT	NOT NULL	—	FK -> cinema_branch(branch_id)	Mã chi nhánh nơi nhân viên làm việc.
employee_code	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_employee_code)	Mã nhân sự (VD: NV-HN01, KT-001).
full_name	VARCHAR(120)	NOT NULL	—	—	Họ và tên nhân viên.
phone	VARCHAR(20)	NOT NULL	—	UNIQUE (uq_employee_phone)	Số điện thoại liên hệ của nhân viên.
role	VARCHAR(20)	NOT NULL	'CASHIER'	CHECK (role IN ('CASHIER', 'MANAGER', 'ACCOUNTANT'))	Vai trò công việc (CASHIER: Thu ngân, MANAGER: Quản lý chi nhánh, ACCOUNTANT: Kế toán).
hired_date	DATE	NOT NULL	—	—	Ngày chính thức nhận việc.

Bảng 11: booking — Đơn đặt vé

Lưu thông tin chung của một lần giao dịch đặt vé xem phim.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
booking_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của đơn đặt vé.
booking_code	VARCHAR(30)	NOT NULL	—	UNIQUE (uq_booking_code)	Mã đơn đặt vé in trên hóa đơn (VD: BK-20261101-0001).
screening_id	INT	NOT NULL	—	FK -> screening(screening_id)	Mã suất chiếu mà đơn đặt vé này mua chỗ.
customer_id	INT	NULL	NULL	FK -> customer(customer_id)	Mã khách hàng thành viên; nhận giá trị NULL khi là **Khách vãng lai mua trực tiếp tại quầy**.
employee_id	INT	NOT NULL	—	FK -> employee(employee_id)	Mã nhân viên thực hiện giao dịch đặt vé.
booked_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	—	Thời điểm khởi tạo đơn đặt vé.
booking_status	VARCHAR(20)	NOT NULL	'CONFIRMED'	CHECK (booking_status IN ('CONFIRMED', 'PENDING', 'CANCELLED'))	Trạng thái đơn (CONFIRMED: Đã xác nhận, PENDING: Chờ thanh toán, CANCELLED: Đã hủy).

Bảng 12: ticket — Vé xem phim chi tiết theo ghế

Lưu từng vé gắn với từng ghế cụ thể trong một đơn đặt vé của một suất chiếu.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
ticket_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh tự tăng của tấm vé.
booking_id	INT	NOT NULL	—	FK -> booking(booking_id)	Mã đơn đặt vé chứa vé này.
screening_id	INT	NOT NULL	—	FK -> screening(screening_id)	Mã suất chiếu của tấm vé (Trigger đảm bảo khớp với booking.screening_id).
seat_id	INT	NOT NULL	—	FK -> seat(seat_id)	Mã ghế vật lý được gán cho vé (Trigger đảm bảo seat.room_id = screening.room_id).
unit_price	DECIMAL(12,2)	NOT NULL	—	CHECK (unit_price >= 0)	Đơn giá bán thực tế chốt tại thời điểm phát hành vé (base_price + surcharge).
ticket_status	VARCHAR(20)	NOT NULL	'VALID'	CHECK (ticket_status IN ('VALID', 'CANCELLED'))	Trạng thái hiệu lực của vé (VALID: Hợp lệ, CANCELLED: Đã hủy).
cancelled_at	DATETIME	NULL	NULL	—	Thời điểm vé bị hủy (NULL nếu vé đang VALID).
active_seat_key	VARCHAR(64)	NULL	Generated	GENERATED ALWAYS AS (CASE WHEN ticket_status = 'VALID' THEN CONCAT(screening_id, '-', seat_id) ELSE NULL END) STORED, UNIQUE (uq_ticket_active_seat)	Cột khóa vật lý sinh tự động bởi DBMS nhằm ngăn chặn tuyệt đối việc bán trùng ghế đang hợp lệ (VALID) trong cùng một suất chiếu và tự động giải phóng ghế khi vé chuyển sang CANCELLED.

Bảng 13: payment — Giao dịch Thanh toán

Lưu lịch sử thanh toán của đơn đặt vé.

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
payment_id	INT AUTO_INCREMENT	NOT NULL	Auto	PRIMARY KEY	Mã định danh giao dịch thanh toán.
booking_id	INT	NOT NULL	—	FK -> booking(booking_id)	Mã đơn đặt vé được thanh toán.
amount	DECIMAL(12,2)	NOT NULL	—	CHECK (amount >= 0)	Số tiền của giao dịch thanh toán (VNĐ).

Tên cột	Kiểu dữ liệu	NULL?	Mặc định	Ràng buộc (PK/FK/UQ/CHECK)	Mô tả ý nghĩa (Tiếng Việt)
payment_method	VARCHAR(20)	NOT NULL	'CASH'	CHECK (payment_method IN ('CASH', 'CARD', 'TRANSFER', 'EWALLET'))	Phương thức thanh toán (CASH: Tiền mặt, CARD: Thẻ ngân hàng, TRANSFER: Chuyển khoản, EWALLET: Ví điện tử).
payment_status	VARCHAR(20)	NOT NULL	'SUCCESS'	CHECK (payment_status IN ('PENDING', 'SUCCESS', 'FAILED', 'REFUNDED'))	Trạng thái thanh toán (PENDING, SUCCESS, FAILED, REFUNDED).
paid_at	DATETIME	NULL	NULL	—	Thời điểm thanh toán hoàn tất (ghi nhận thời điểm tính doanh thu khi payment_status = 'SUCCESS').
transaction_ref	VARCHAR(60)	NULL	NULL	—	Mã tham chiếu biên lai / ngân hàng / cổng thanh toán.
active_success_booking	INT	NULL	Generated	GENERATED ALWAYS AS (CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END) STORED, UNIQUE (uq_payment_one_success)	Cột sinh tự động đảm bảo mỗi đơn đặt vé chỉ có tối đa **01** giao dịch thanh toán ở trạng thái SUCCESS, ngăn chặn thanh toán đúp và nhân bản doanh thu.

3. Tổng hợp Ràng buộc Tham chiếu Khóa ngoại (Foreign Key Matrix)

Tên Ràng buộc FK	Bảng Con (Child Table)	Cột FK	Bảng Cha (Parent Table)	Cột PK Tham chiếu	Quy tắc ON DELETE / ON UPDATE
fk_room_branch	screening_room	branch_id	cinema_branch	branch_id	RESTRICT / CASCADE
fk_seat_room	seat	room_id	screening_room	room_id	RESTRICT / CASCADE
fk_seat_type	seat	seat_type_code	seat_type	seat_type_code	RESTRICT / CASCADE
fk_mg_movie	movie_genre	movie_id	movie	movie_id	CASCADE / CASCADE
fk_mg_genre	movie_genre	genre_id	genre	genre_id	RESTRICT / CASCADE
fk_screening_movie	screening	movie_id	movie	movie_id	RESTRICT / CASCADE
fk_screening_room	screening	room_id	screening_room	room_id	RESTRICT / CASCADE
fk_employee_branch	employee	branch_id	cinema_branch	branch_id	RESTRICT / CASCADE

Tên Ràng buộc FK	Bảng Con (Child Table)	Cột FK	Bảng Cha (Parent Table)	Cột PK Tham chiếu	Quy tắc ON DELETE / ON UPDATE
fk_booking_screening	booking	screening_id	screening	screening_id	RESTRICT / CASCADE
fk_booking_customer	booking	customer_id	customer	customer_id	SET NULL / CASCADE
fk_booking_employee	booking	employee_id	employee	employee_id	RESTRICT / CASCADE
fk_ticket_booking	ticket	booking_id	booking	booking_id	RESTRICT / RESTRICT
fk_ticket_booking_scr	ticket	(booking_id, screening_id)	booking	(booking_id, screening_id)	RESTRICT / RESTRICT
fk_ticket_screening	ticket	screening_id	screening	screening_id	RESTRICT / RESTRICT
fk_ticket_seat	ticket	seat_id	seat	seat_id	RESTRICT / RESTRICT
fk_payment_booking	payment	booking_id	booking	booking_id	RESTRICT / RESTRICT

CHƯƠNG 4. PHỤ THUỘC HÀM VÀ CHỨNG MINH CHUẨN HÓA 3NF (TIÊU CHÍ R4)

(Tài liệu đáp ứng Tiêu chí R4 — 1.0 điểm)

1. Cơ sở Lý thuyết Chuẩn hóa Quan hệ

Cho lược đồ quan hệ R(U) với tập thuộc tính U và tập phụ thuộc hàm F xác định trên U:

• **Khóa dự tuyển (Candidate Key – K):** Tập con K ⊆ U sao cho bao đóng K^+_F = U và không tồn tại tập con thực sự K' ⊂ K nào có (K')^+_F = U.

• **Thuộc tính khóa (Prime Attribute):** Thuộc tính thuộc về ít nhất một khóa dự tuyển của R.

• **Thuộc tính không khóa (Non-prime Attribute):** Thuộc tính không thuộc bất kỳ khóa dự tuyển nào của R.

• **Dạng chuẩn 1 (1NF):** Mọi thuộc tính trong R đều mang giá trị nguyên tố (atomic), không có thuộc tính đa trị hoặc thuộc tính lặp.

• **Dạng chuẩn 2 (2NF):** R đạt 1NF và mọi thuộc tính không khóa đều phụ thuộc hàm đầy đủ vào mọi khóa dự tuyển của R (không tồn tại phụ thuộc hàm bộ phận X → A với X ⊂ K thực sự và A không khóa).

• **Dạng chuẩn 3 (3NF):** R đạt 2NF và với mọi phụ thuộc hàm không tầm thường X → A ∈ F^+ (trong đó A ∉ X), thì hoặc:

1. X là một siêu khóa (Superkey) của R, **hoặc**
2. A là một thuộc tính khóa (Prime attribute) của R.

(Nói cách khác, không tồn tại thuộc tính không khóa nào phụ thuộc bắc cầu vào khóa dự tuyển).

2. Các Quyết định Thiết kế Loại bỏ Vi phạm Chuẩn hóa Ngay từ Đầu

Trước khi đi vào chứng minh từng bảng, kiến trúc sư CSDL đã chủ động loại bỏ 5 nguy cơ vi phạm 2NF/3NF thường gặp trong bài toán rạp chiếu phim:

1. **Không lưu cột capacity (sức chứa) trong screening_room:** Sức chứa của phòng bằng tổng số dòng ghế thuộc phòng đó trong bảng `seat`. Việc tính động qua truy vấn/View giúp loại bỏ hoàn toàn dị thường cập nhật khi thêm/bớt ghế.

2. **Không lưu cột total_amount trong booking:** Tổng tiền đơn vé bằng tổng `unit_price` của các vé thuộc đơn đó. Không lưu dư thừa trong `booking` giúp tránh mâu thuẫn dữ liệu giữa đơn và danh sách vé.

3. **Không lưu cột room_id trong ticket:** Nếu lưu `room_id` trong `ticket`, sẽ xuất hiện phụ thuộc bắc cầu `ticket_id -> screening_id -> room_id` và `ticket_id -> seat_id -> room_id` (vi phạm 3NF). Thay vào đó, ràng buộc "ghế trên vé phải thuộc đúng phòng chiếu của suất chiếu" được cưỡng chế bằng Trigger `trg_ticket_validate_ins/upd`.

4. **Bản chất lịch sử của ticket.unit_price:** Tại thời điểm phát hành vé, đơn giá được tính từ `screening.base_price + seat_type.surcharge` tại thời điểm đó và chốt cứng vào `ticket.unit_price`. Về sau, khi rạp thay đổi bảng giá hạng ghế (`seat_type.surcharge`) hoặc điều chỉnh giá cơ sở cho các đợt khuyến mãi mới, giá trên các vé đã bán trong quá khứ **không được phép thay đổi**. Do đó, trên tập dữ liệu lịch sử của bảng `ticket`, `unit_price` là thuộc tính phụ thuộc trực tiếp vào khóa `ticket_id` (ticket_id → unit_price), không phụ thuộc hàm vào giá hiện hành của `seat_type` hay `screening`.

5. **Bản chất vận hành của screening.end_time và các cột sinh vật lý (GENERATED ALWAYS AS ... STORED):**

• `screening.end_time` bao gồm thời lượng phim cộng thêm thời gian chiếu trailer quảng cáo và thời gian nhân viên vệ sinh phòng chiếu (có thể linh hoạt 15–30 phút tùy khung giờ cao điểm hay suất IMAX). Vì

vậy `end_time` không suy diễn hàm thuần túy từ `(start_time, movie_id)`, mà chỉ chịu ràng buộc miền giá trị end_time ≥ start_time + duration_minutes.

• Hai cột `active_seat_key` (trong `ticket`) và `active_success_booking` (trong `payment`) là biểu thức chỉ mục có điều kiện do DBMS tự động tính toán (`GENERATED ALWAYS AS ... STORED`) để thực thi ràng buộc duy nhất một phần (Partial Unique Index) trong MySQL 8.4. Người dùng và ứng dụng không thể `INSERT`/`UPDATE` trực tiếp hai cột này nên tuyệt đối không thể xảy ra dị thường cập nhật (Update Anomaly).

3. Phân tích Phụ thuộc Hàm, Tìm Khóa và Chứng minh 3NF cho Từng Bảng

3.1. Quan hệ `cinema_branch`

• **Tập thuộc tính:** U_1 = {branch_id, branch_code, branch_name, address, city, phone, active}

• **Tập phụ thuộc hàm F_1:**

• f_{1.1}: branch_id → {branch_code, branch_name, address, city, phone, active}

• f_{1.2}: branch_code → {branch_id, branch_name, address, city, phone, active}

• f_{1.3}: branch_name → {branch_id, branch_code, address, city, phone, active}

• f_{1.4}: phone → {branch_id, branch_code, branch_name, address, city, active}

• **Bao đóng & Khóa dự tuyển:**

• (branch_id)^+ = U_1; (branch_code)^+ = U_1; (branch_name)^+ = U_1; (phone)^+ = U_1.

• Các khóa dự tuyển (1 thuộc tính): K_{1.1} = {branch_id} (PK), K_{1.2} = {branch_code}, K_{1.3} = {branch_name}, K_{1.4} = {phone}.

• **Kiểm tra dạng chuẩn:**

• **1NF:** Mọi thuộc tính đều nguyên tố ⇒ Đạt **1NF**.

• **2NF:** Mọi khóa dự tuyển đều gồm 1 thuộc tính duy nhất, không thể có phụ thuộc bộ phận ⇒ Đạt **2NF**.

• **3NF:** Mọi phụ thuộc hàm không tầm thường trong F_1 đều có vế trái là một khóa dự tuyển (siêu khóa) ⇒ Đạt **3NF** (và đạt cả **BCNF**).

3.2. Quan hệ `screening_room`

• **Tập thuộc tính:** U_2 = {room_id, branch_id, room_name, room_type, active}

• **Tập phụ thuộc hàm F_2:**

• f_{2.1}: room_id → {branch_id, room_name, room_type, active}

• f_{2.2}: {branch_id, room_name} → {room_id, room_type, active}

• **Bao đóng & Khóa dự tuyển:**

• (room_id)^+ = U_2; ({branch_id, room_name})^+ = U_2.

• Hai khóa dự tuyển: K_{2.1} = {room_id} (PK) và K_{2.2} = {branch_id, room_name} (UQ).

• Thuộc tính khóa: {room_id, branch_id, room_name}; Thuộc tính không khóa: {room_type, active}.

• **Kiểm tra dạng chuẩn:**

• **1NF:** Tất cả thuộc tính đều nguyên tố ⇒ Đạt **1NF**.

• **2NF:** Xét khóa phức hợp K_{2.2} = {branch_id, room_name}: chỉ mình `branch_id` không xác định được `room_type` hay `active` (một chi nhánh có nhiều phòng khác loại), và chỉ mình `room_name` (VD: `Phòng 01`) cũng không xác định được `room_type` hay `active` vì các chi nhánh khác nhau đều có `Phòng 01`. Do đó các thuộc tính không khóa phụ thuộc đầy đủ vào cả hai khóa dự tuyển ⇒ Đạt **2NF**.

• **3NF:** Cả f_{2.1} và f_{2.2} đều có vế trái là khóa dự tuyển ⇒ Đạt **3NF**.

3.3. Quan hệ `seat_type`

• **Tập thuộc tính:** U_3 = {seat_type_code, type_name, surcharge}

• **Tập phụ thuộc hàm F_3:**

• f_{3.1}: seat_type_code → {type_name, surcharge}

• f_{3.2}: type_name → {seat_type_code, surcharge}

• **Khóa dự tuyển:** K_{3.1} = {seat_type_code} (PK), K_{3.2} = {type_name} (UQ).

• **Kiểm tra dạng chuẩn:** Mọi thuộc tính nguyên tố (1NF), các khóa đều đơn thuộc tính (2NF), mọi FD đều có vế trái là khóa dự tuyển ⇒ Đạt **3NF**.

3.4. Quan hệ `seat`

• **Tập thuộc tính:** U_4 = {seat_id, room_id, seat_row, seat_number, seat_type_code}

• **Tập phụ thuộc hàm F_4:**

• f_{4.1}: seat_id → {room_id, seat_row, seat_number, seat_type_code}

• f_{4.2}: {room_id, seat_row, seat_number} → {seat_id, seat_type_code}

• **Bao đóng & Khóa dự tuyển:**

• (seat_id)^+ = U_4; ({room_id, seat_row, seat_number})^+ = U_4.

• Khóa dự tuyển: K_{4.1} = {seat_id} (PK) và K_{4.2} = {room_id, seat_row, seat_number} (UQ).

• Thuộc tính không khóa duy nhất: `seat_type_code` (vì cấu hình hạng ghế theo hàng có thể khác nhau giữa phòng 2D, 3D và IMAX hoặc ghế đôi ở cuối phòng nên cần đủ bộ `{room_id, seat_row, seat_number}`).

• **Kiểm tra dạng chuẩn:**

• **1NF:** Giá trị đơn nguyên tố ⇒ Đạt **1NF**.

• **2NF:** `seat_type_code` phụ thuộc hàm đầy đủ vào K_{4.1} và K_{4.2} ⇒ Đạt **2NF**.

• **3NF:** Mọi FD không tầm thường trong F_4 đều có vế trái là siêu khóa; bảng `seat` không lưu `surcharge` hay `type_name` nên không có phụ thuộc bắc cầu qua `seat_type_code` ⇒ Đạt **3NF**.

3.5. Quan hệ `movie`

• **Tập thuộc tính:** U_5 = {movie_id, movie_code, title, duration_minutes, release_date, age_rating, country, description}

• **Tập phụ thuộc hàm F_5:**

• f_{5.1}: movie_id → U_5 \ {movie_id}

• f_{5.2}: movie_code → U_5 \ {movie_code}

• **Khóa dự tuyển:** K_{5.1} = {movie_id} (PK), K_{5.2} = {movie_code} (UQ).

• **Kiểm tra dạng chuẩn:** Thể loại phim đa trị đã được tách sang bảng `movie_genre` nên `movie` đạt **1NF**; các khóa đều đơn thuộc tính nên đạt **2NF**; không có phụ thuộc bắc cầu giữa các thuộc tính không khóa ⇒ Đạt **3NF**.

3.6. Quan hệ `genre`

• **Tập thuộc tính:** U_6 = {genre_id, genre_name, description}

• **Tập phụ thuộc hàm F_6:**

• f_{6.1}: genre_id → {genre_name, description}

• f_{6.2}: genre_name → {genre_id, description}

• **Khóa dự tuyển:** K_{6.1} = {genre_id} (PK), K_{6.2} = {genre_name} (UQ).

• **Kiểm tra dạng chuẩn:** Mọi FD có vế trái là khóa dự tuyển ⇒ Đạt **1NF, 2NF, 3NF**.

3.7. Quan hệ `movie_genre` (Khóa chính phức hợp)

• **Tập thuộc tính:** U_7 = {movie_id, genre_id}

• **Tập phụ thuộc hàm F_7:** Chỉ có các phụ thuộc hàm tầm thường ({movie_id, genre_id} → movie_id, genre_id).

• **Khóa dự tuyển duy nhất:** K_7 = {movie_id, genre_id} (PK phức hợp).

• **Kiểm tra dạng chuẩn:**

• **1NF:** Mỗi ô chứa đúng một mã nguyên tố ⇒ Đạt **1NF**.

• **2NF & 3NF:** Quan hệ hoàn toàn không có thuộc tính không khóa (U_7 \ K_7 = ∅), do đó không thể tồn tại phụ thuộc bộ phận hay phụ thuộc bắc cầu ⇒ Đạt **2NF, 3NF, BCNF**.

3.8. Quan hệ `screening`

• **Tập thuộc tính:** U_8 = {screening_id, movie_id, room_id, start_time, end_time, base_price, status}

• **Tập phụ thuộc hàm F_8:**

• f_{8.1}: screening_id → {movie_id, room_id, start_time, end_time, base_price, status}

• **Khóa dự tuyển:** K_8 = {screening_id} (PK). (Lưu ý: một phòng chiếu có thể có một suất bị `CANCELLED` rồi tạo lại suất `OPEN` mới ở cùng khung giờ nên `(room_id, start_time)` không phải khóa cứng trên toàn tập lịch sử mà chỉ là ràng buộc không giao nhau trên các suất `OPEN`).

• **Kiểm tra dạng chuẩn:**

• **1NF & 2NF:** Khóa đơn `screening_id`, các thuộc tính nguyên tố ⇒ Đạt **1NF, 2NF**.

• **3NF:** Như đã phân tích tại Mục 2.5, `end_time` bao gồm thời gian quảng cáo và dọn phòng tùy suất chiếu, và `base_price` được ấn định theo khung giờ vàng/ngày lễ cho từng suất chiếu chứ không phụ thuộc hàm vào `movie_id` hay `room_id`. Bảng không lưu `branch_id` hay `title` ⇒ Không có phụ thuộc bắc cầu ⇒ Đạt **3NF**.

3.9. Quan hệ `customer`

• **Tập thuộc tính:** U_9 = {customer_id, full_name, phone, email, birth_date, created_at}

• **Tập phụ thuộc hàm F_9:**

• f_{9.1}: customer_id → {full_name, phone, email, birth_date, created_at}

• f_{9.2}: phone → {customer_id, full_name, email, birth_date, created_at}

• f_{9.3}: email → {customer_id, full_name, phone, birth_date, created_at}

• **Khóa dự tuyển:** K_{9.1} = {customer_id} (PK), K_{9.2} = {phone} (UQ), K_{9.3} = {email} (UQ).

• **Kiểm tra dạng chuẩn:** Mọi FD không tầm thường đều có vế trái là khóa dự tuyển ⇒ Đạt **1NF, 2NF, 3NF**.

3.10. Quan hệ `employee`

• **Tập thuộc tính:** U_{10} = {employee_id, branch_id, employee_code, full_name, phone, role, hired_date}

• **Tập phụ thuộc hàm F_{10}:**

• f_{10.1}: employee_id → {branch_id, employee_code, full_name, phone, role, hired_date}

• f_{10.2}: employee_code → {employee_id, branch_id, full_name, phone, role, hired_date}

• f_{10.3}: phone → {employee_id, branch_id, employee_code, full_name, role, hired_date}

• **Khóa dự tuyển:** K_{10.1} = {employee_id} (PK), K_{10.2} = {employee_code} (UQ), K_{10.3} = {phone} (UQ).

• **Kiểm tra dạng chuẩn:** Bảng không lưu thông tin chi nhánh (`branch_name`, `city`), mọi FD đều xuất phát từ khóa dự tuyển ⇒ Đạt **1NF, 2NF, 3NF**.

3.11. Quan hệ `booking`

• **Tập thuộc tính:** U_{11} = {booking_id, booking_code, screening_id, customer_id, employee_id, booked_at, booking_status}

• **Tập phụ thuộc hàm F_{11}:**

• f_{11.1}: booking_id → {booking_code, screening_id, customer_id, employee_id, booked_at, booking_status}

• f_{11.2}: booking_code → {booking_id, screening_id, customer_id, employee_id, booked_at, booking_status}

• **Khóa dự tuyển:** K_{11.1} = {booking_id} (PK), K_{11.2} = {booking_code} (UQ).

• **Kiểm tra dạng chuẩn:** Danh sách ghế và vé đã được tách sang bảng `ticket` (đạt **1NF**); khóa đơn thuộc tính (đạt **2NF**); không lưu tổng tiền hay thông tin khách hàng/phim dư thừa ⇒ Đạt **3NF**.

3.12. Quan hệ `ticket`

• **Tập thuộc tính logic:** U_{12} = {ticket_id, booking_id, screening_id, seat_id, unit_price, ticket_status, cancelled_at}

• **Tập phụ thuộc hàm F_{12}:**

• f_{12.1}: ticket_id → {booking_id, screening_id, seat_id, unit_price, ticket_status, cancelled_at}

• f_{12.2}: booking_id → screening_id (mỗi đơn đặt vé thuộc một suất chiếu xác định; cột `screening_id` có mặt trong `ticket` để hỗ trợ chỉ mục vật lý chống bán trùng ghế `(screening_id, seat_id)`)

• **Thảo luận học thuật và chuẩn hóa ticket:**

• Xét phụ thuộc f_{12.2}: booking_id → screening_id: Trong thiết kế chuẩn hóa thuần túy, `screening_id` có thể suy ra từ `booking_id`. Tuy nhiên, trong hệ quản trị MySQL 8.4, chỉ mục `UNIQUE` không thể tham chiếu cột của bảng khác (`booking.screening_id`). Để DBMS trực tiếp cưỡng chế quy tắc cốt lõi "không bao giờ bán 2 vé hợp lệ cho cùng một ghế trong cùng một suất chiếu" ở mức chỉ mục phần cứng B-Tree mà không phụ thuộc độc quyền vào mã Java, `(booking_id, screening_id)` tạo thành tham chiếu khóa tổng hợp (composite foreign key reference) sang `booking(booking_id, screening_id)` và được khóa chặt bởi Trigger `NEW.screening_id = booking.screening_id`.

• Đối với `unit_price`: Đây là giá chốt lịch sử tại thời điểm giao dịch (ticket_id → unit_price), không phụ thuộc hàm vào giá hiện tại của `screening` hay `seat_type`.

• Đối với `active_seat_key`: Là cột vật lý `GENERATED ALWAYS AS (...) STORED` do hệ quản trị tự duy trì để đánh chỉ mục duy nhất có điều kiện cho các vé `VALID`.

• Do đó, mọi thuộc tính nghiệp vụ của `ticket` đều nguyên tố (**1NF**), phụ thuộc đầy đủ vào khóa đơn `ticket_id` (**2NF**), và không phát sinh bất kỳ dị thường thêm/sửa/xóa nào (**3NF**).

3.13. Quan hệ `payment`

• **Tập thuộc tính logic:** U_{13} = {payment_id, booking_id, amount, payment_method, payment_status, paid_at, transaction_ref}

• **Tập phụ thuộc hàm F_{13}:**

• f_{13.1}: payment_id → {booking_id, amount, payment_method, payment_status, paid_at, transaction_ref}

• **Khóa dự tuyển:** K_{13} = {payment_id} (PK). (Một `booking_id` có thể có nhiều lần thử thanh toán `FAILED` trước khi `SUCCESS` nên `booking_id` không phải khóa trên toàn bảng `payment`, chỉ duy nhất trên tập con `payment_status = 'SUCCESS'` thông qua cột chỉ mục sinh `active_success_booking`).

• **Kiểm tra dạng chuẩn:**

• **1NF:** Mọi thuộc tính đều nguyên tố ⇒ Đạt **1NF**.

• **2NF:** Khóa dự tuyển đơn `payment_id` ⇒ Đạt **2NF**.

• **3NF:** Không có phụ thuộc hàm bắc cầu giữa các thuộc tính không khóa ⇒ Đạt **3NF**.

4. Bảng Tổng kết Kết quả Chuẩn hóa 13 Quan hệ

STT	Tên Quan hệ	Khóa chính (PK)	Khóa dự tuyển khác (Alternate Keys)	1NF	2NF	3NF	Kết luận
1	`cinema_branch`	`branch_id`	`branch_code`, `branch_name`, `phone`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
2	`screening_room`	`room_id`	`(branch_id, room_name)`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
3	`seat_type`	`seat_type_code`	`type_name`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
4	`seat`	`seat_id`	`(room_id, seat_row, seat_number)`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
5	`movie`	`movie_id`	`movie_code`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
6	`genre`	`genre_id`	`genre_name`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
7	`movie_genre`	`(movie_id, genre_id)`	—	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
8	`screening`	`screening_id`	—	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
9	`customer`	`customer_id`	`phone`, `email`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
10	`employee`	`employee_id`	`employee_code`, `phone`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
11	`booking`	`booking_id`	`booking_code`	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
12	`ticket`	`ticket_id`	`active_seat_key` (khi VALID)	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**
13	`payment`	`payment_id`	`active_success_booking` (khi SUCCESS)	Đạt	Đạt	Đạt	**Đạt chuẩn 3NF**

CHƯƠNG 5. KIẾN TRÚC ỨNG DỤNG, GIAO DỊCH ĐỒNG THỜI & CÁC MÀN HÌNH NGHIỆP VỤ (R5, R6, R7)

5.1. Kiến trúc Phân tầng Nghiêm ngặt & Kiểm soát Giao dịch ACID trên MySQL 8.4 LTS

Ứng dụng Desktop được xây dựng trên nền tảng **Java 21 LTS**, **Java Swing + FlatLaf 3.5**, **JDBC (MySQL Connector/J 9.1.0 + PreparedStatement)** kết nối trực tiếp tới **MySQL Community Server 8.4.0 LTS** (`InnoDB`, `utf8mb4_unicode_ci`, `ONLY_FULL_GROUP_BY`, `STRICT_TRANS_TABLES`) và **Apache PDFBox 3**, tuân thủ nghiêm ngặt kiến trúc phân tầng (`ui` → `service` → `repository` → `shared.db`). Bộ kiểm thử kiến trúc tự động `ArchitectureComplianceTest` (ArchUnit) đảm bảo không một lớp giao diện nào được phép truy cập trực tiếp `java.sql.*`. Mọi kết nối vận hành đều sử dụng tài khoản phân quyền tối thiểu `cinema_app` (mật khẩu chỉ lưu trong tệp cấu hình cục bộ `config/application.properties` hoặc biến môi trường, tuyệt đối không mã hóa cứng).

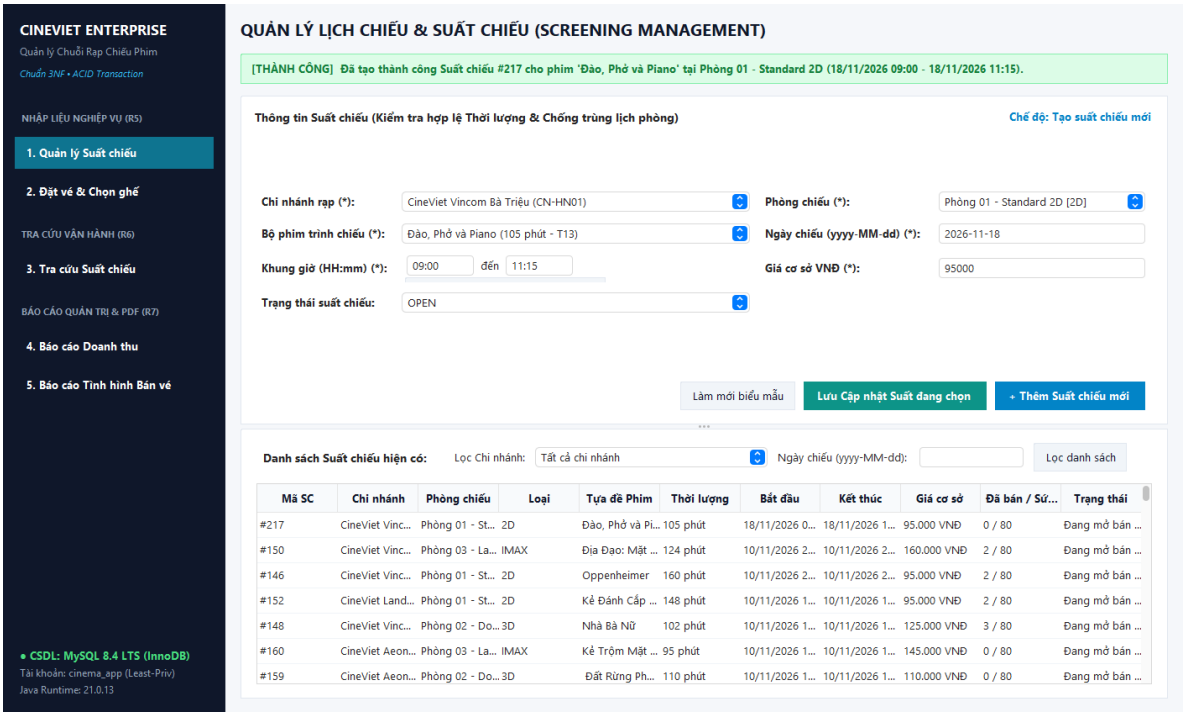
Đối với hai bài toán tranh chấp đồng thời cốt lõi trên MySQL 8.4 InnoDB:

• **Chống chồng lấn lịch chiếu đồng thời (BR-06):** `ScreeningService` khóa bi quan dòng phòng chiếu `SELECT room_id, active FROM screening_room WHERE room_id = ? FOR UPDATE` trước khi kiểm tra giao thoa khung giờ, kết hợp với Trigger `trg_screening_validate_ins/upd` tại tầng MySQL Server.

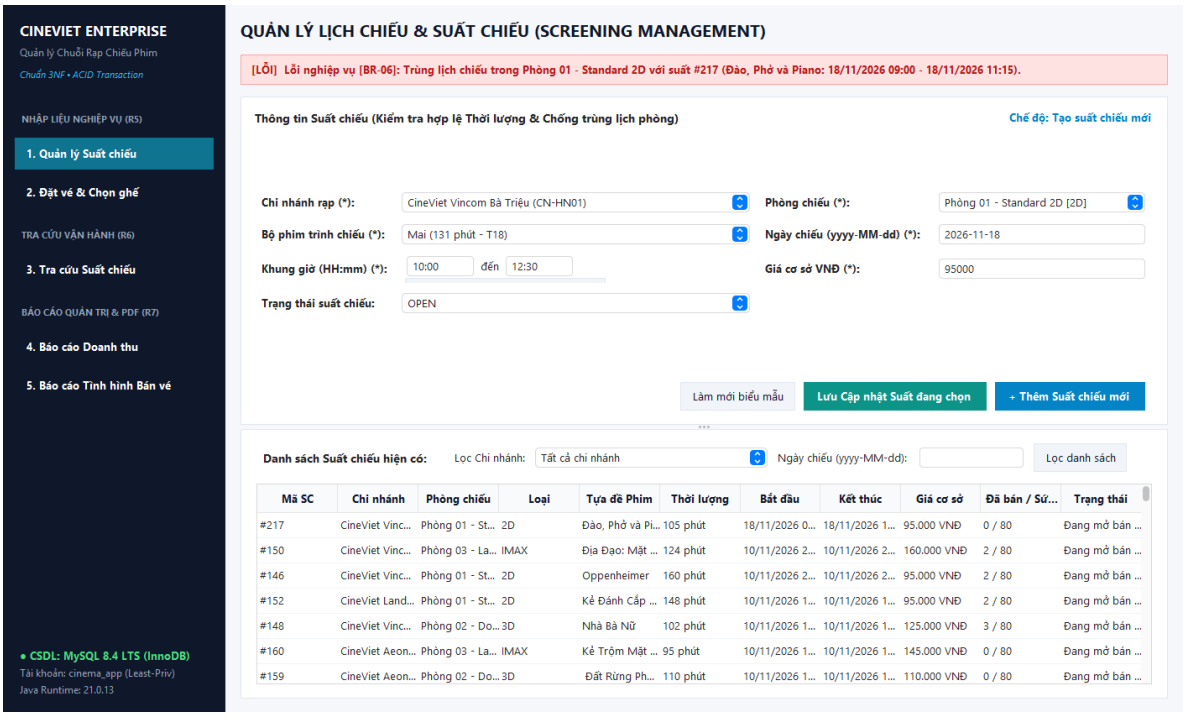
• **Chống bán trùng ghế đồng thời (BR-10):** `BookingService` khóa dòng suất chiếu `SELECT ... FROM screening WHERE screening_id = ? FOR UPDATE`, kiểm tra trạng thái ghế, lưu `booking` + `ticket` + `payment` nguyên tử và được bảo vệ lớp cuối bởi chỉ mục duy nhất trên cột sinh lưu trữ `uq_ticket_active_seat (active_seat_key)`.

5.2. Màn hình Nhập liệu 1: Quản lý Lịch chiếu & Suất chiếu (Đáp ứng R5)

Màn hình cho phép Quản lý Chi nhánh tạo mới, cập nhật hoặc hủy suất chiếu; tự động gợi ý giờ kết thúc dựa trên thời lượng bộ phim cộng 20 phút dọn phòng; kiểm tra hợp lệ giá cơ sở dương, thời lượng tối thiểu (`BR-05`) và từ chối tức thì mọi suất chiếu trùng lịch trong cùng một phòng chiếu (`BR-06`).



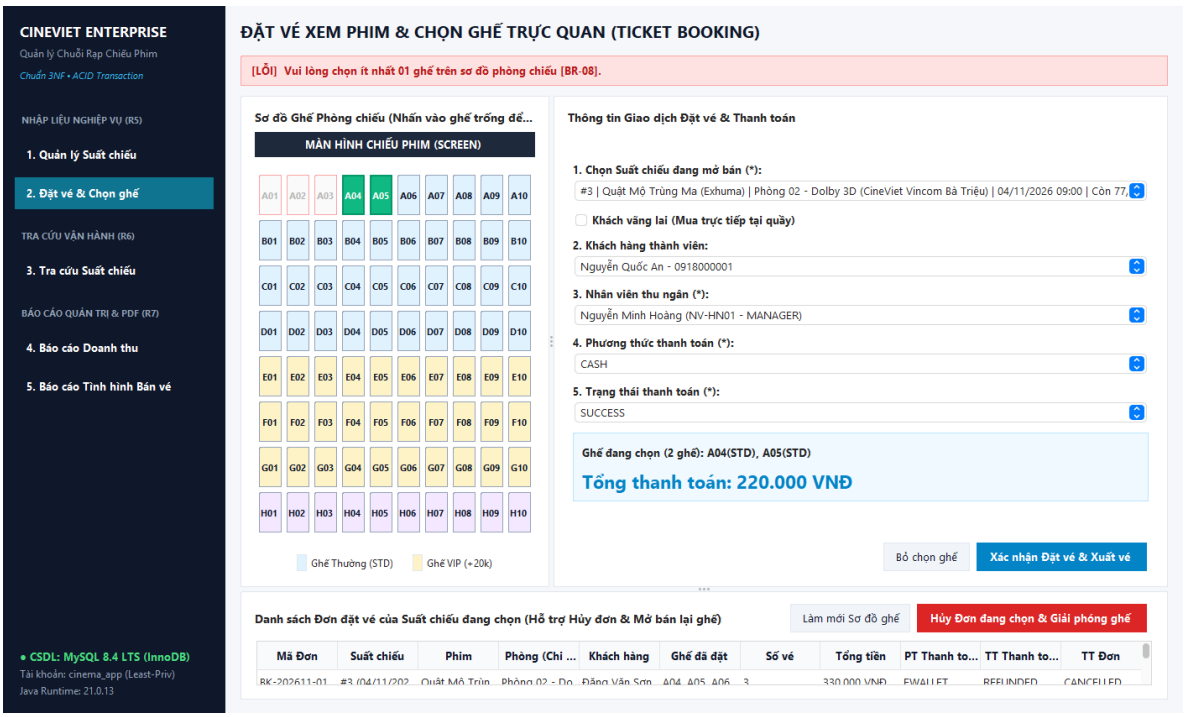
Hình 5.1. Màn hình Quản lý Suất chiếu — Tạo mới thành công suất chiếu hợp lệ trên MySQL 8.4 LTS.



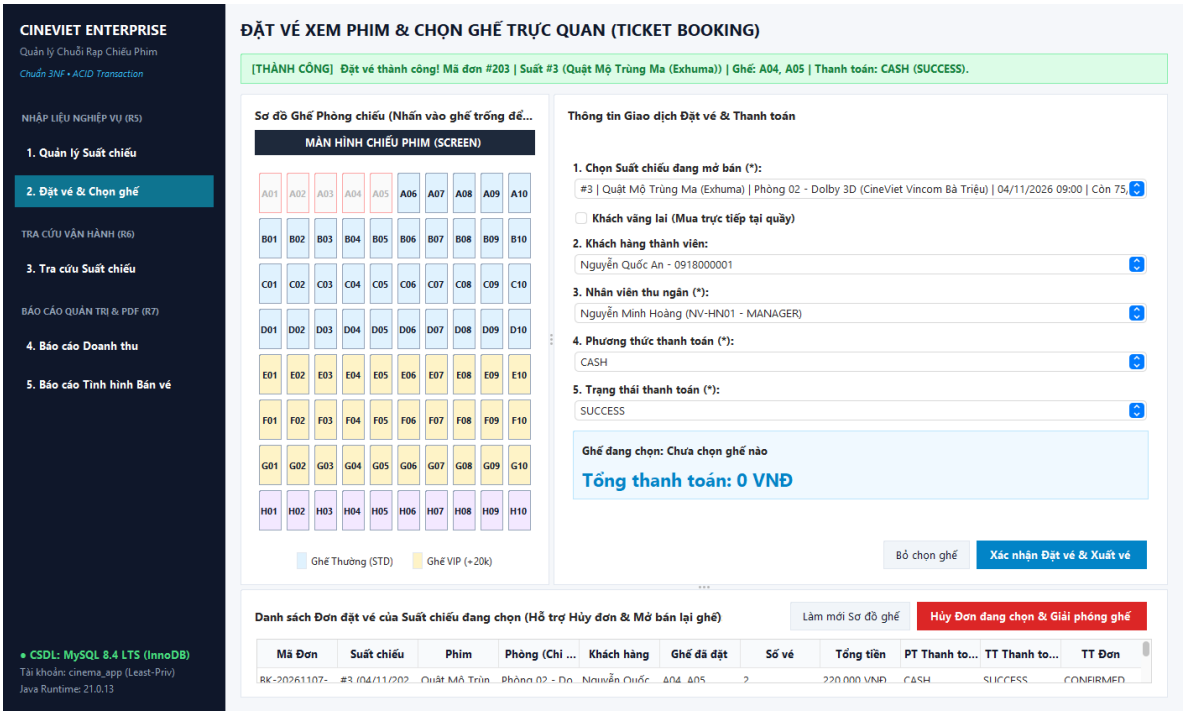
Hình 5.2. Màn hình Quản lý Suất chiếu — Phát hiện và từ chối suất chiếu chồng lấn thời gian trong cùng phòng [BR-06].

5.3. Màn hình Nhập liệu 2: Đặt vé & Chọn ghế theo Sơ đồ Phòng chiếu (Đáp ứng R5)

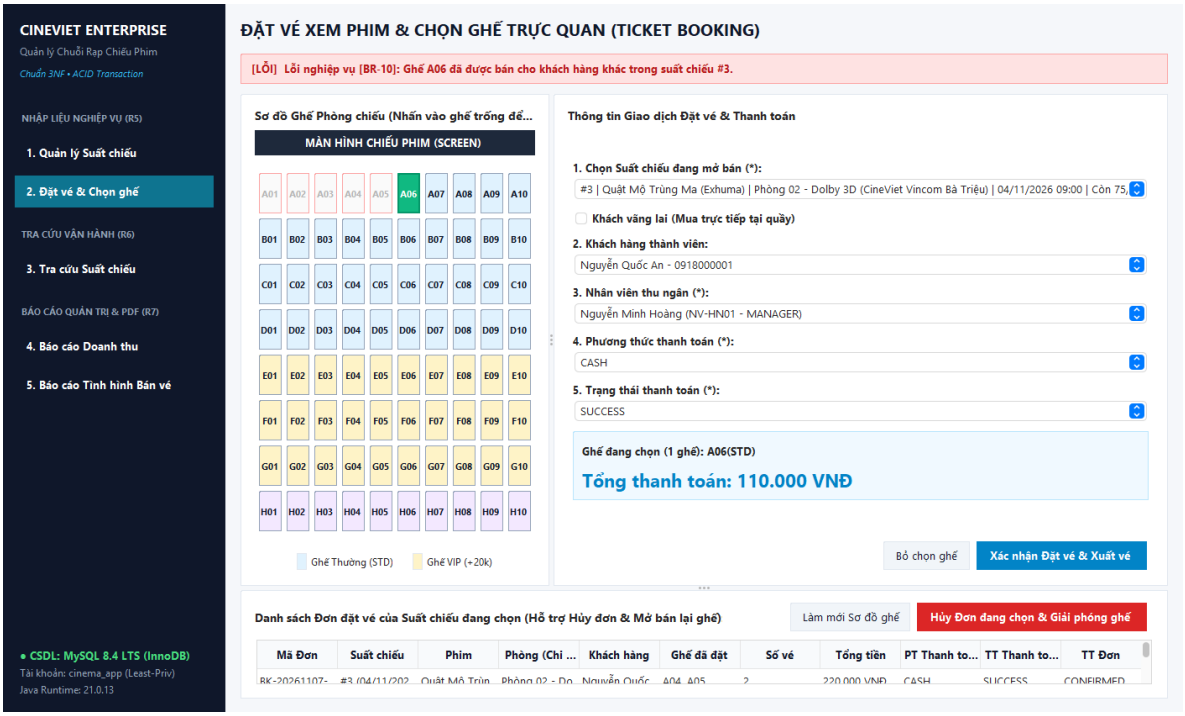
Màn hình phục vụ Thu ngân bán vé cho cả **Khách hàng thành viên** lẫn **Khách vãng lai tại quầy**, hiển thị trực quan sơ đồ 80 ghế (8 hàng A–H × 10 ghế) phân biệt rõ hạng ghế Thường (`STD`), `VIP`, Ghế đôi (`COUPLE`), ghế đang chọn và ghế đã bán; tính tự động đơn giá lịch sử (`base_price + surcharge`), xử lý thanh toán nguyên tử và cho phép **Hủy đơn đặt vé để giải phóng ghế mở bán lại (BR-11)**.



Hình 5.3. Màn hình Đặt vé & Chọn ghế — Chọn nhiều ghế trống trực tiếp trên sơ đồ phòng chiếu và tính tổng tiền tự động.



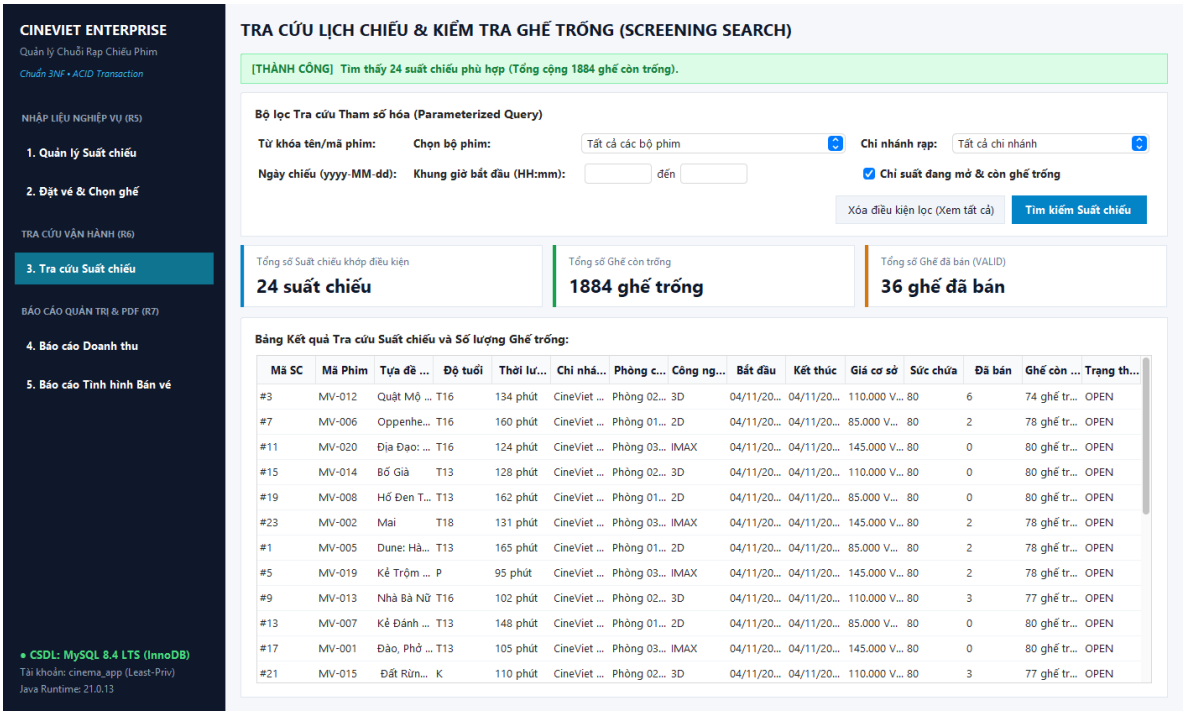
Hình 5.4. Màn hình Đặt vé & Chọn ghế — Xác nhận giao dịch đặt vé và thanh toán thành công trên MySQL 8.4 LTS.



Hình 5.5. Màn hình Đặt vé & Chọn ghế — Chặn giao dịch khi ghế vừa bị quầy thu ngân khác đặt trước [BR-10].

5.4. Màn hình Tìm kiếm Vận hành: Tra cứu Suất chiếu & Kiểm tra Ghế trống (Đáp ứng R6)

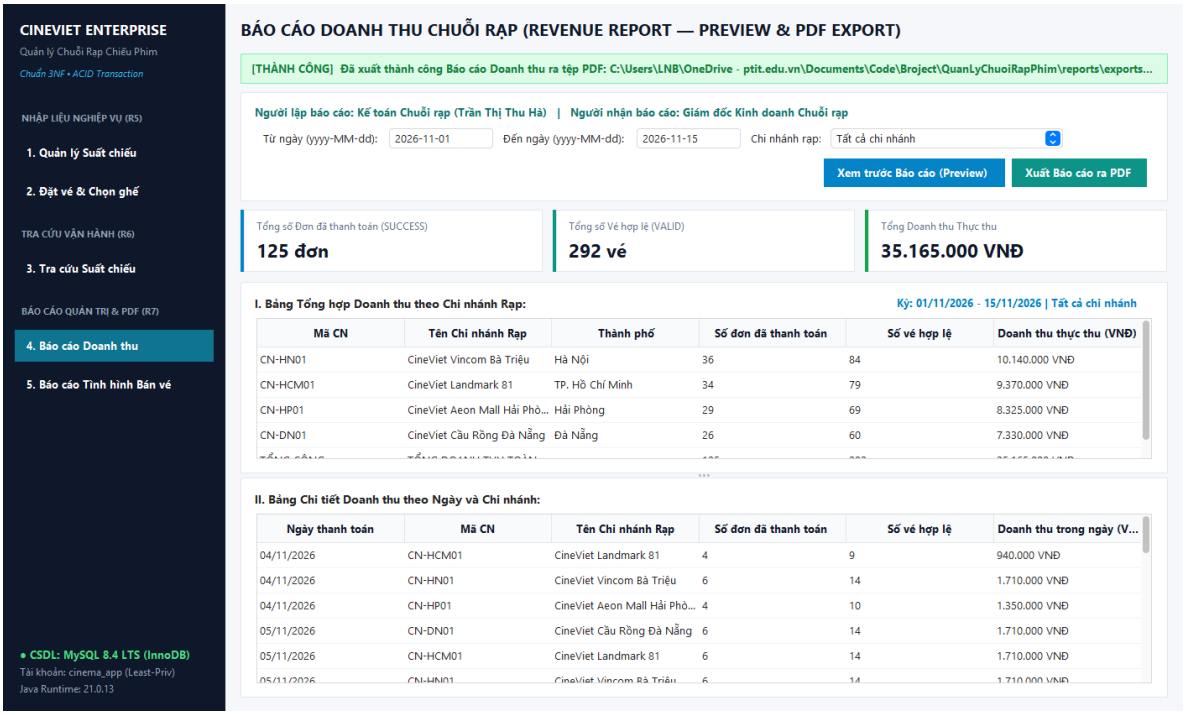
Màn hình sử dụng truy vấn tham số hóa (`PreparedStatement`) trên View `vw_screening_search` của MySQL 8.4, cho phép lọc linh hoạt theo từ khóa tên/mã phim, bộ phim, chi nhánh rạp, ngày chiếu, khung giờ bắt đầu và tùy chọn chỉ hiển thị suất còn ghế trống.



Hình 5.6. Màn hình Tra cứu Suất chiếu — Hiển thị danh sách suất chiếu khớp điều kiện kèm sức chứa, vé đã bán và số ghế còn trống.

5.5. Báo cáo Thực tiễn 1: Báo cáo Doanh thu Chuỗi rạp — Preview & Xuất PDF (Đáp ứng R7)

• **Người lập báo cáo:** Kế toán Chuỗi rạp (`Trần Thị Thu Hà`).
• **Người nhận báo cáo:** Giám đốc Kinh doanh Chuỗi rạp.
• **Quy tắc ghi nhận doanh thu (BR-14, BR-15):** Chỉ tính tổng `ticket.unit_price` của các vé `VALID` thuộc đơn `CONFIRMED` có giao dịch thanh toán `SUCCESS` (sử dụng View `vw_verified_booking_revenue` và ràng buộc `uq_payment_one_success` trên MySQL 8.4 để triệt tiêu hoàn toàn nguy cơ nhân đôi doanh thu khi một đơn có lịch sử thanh toán lỗi `FAILED` trước khi thanh toán lại `SUCCESS`). Tổng doanh thu thực thu trên tập dữ liệu mẫu đạt chính xác **35.165.000 VNĐ** (khớp tuyệt đối 100% với truy vấn SQL đối soát độc lập).



Hình 5.7. Màn hình Xem trước (Preview) Báo cáo Doanh thu Chuỗi rạp kết xuất trực tiếp từ MySQL 8.4 LTS.



HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM CINEVIET

BÁO CÁO DOANH THU CHUỖI RẠP THEO KỲ HẠN VÀ CHI NHÁNH

Kỳ báo cáo: Từ ngày 01/11/2026 đến ngày 15/11/2026 | Phạm vi chi nhánh: Tất cả chi nhánh

Người lập báo cáo: Trần Thị Thu Hà (Mã NV: NV-HN02) (Kế toán Chuỗi rạp) | Người nhận báo cáo: Giám đốc Kinh doanh Chuỗi rạp | Thời điểm lập: 07/11/2026 13:15

TỔNG HỢP TOÀN KỲ: Tổng số đơn đã thanh toán: 125 đơn | Tổng số vé hợp lệ: 292 vé | TỔNG DOANH THU THỰC THU: 35.165.000 VNĐ

Quy tắc ghi nhận doanh thu: Chỉ tính vé hợp lệ (VALID) thuộc các đơn đã xác nhận (CONFIRMED) và có giao dịch thanh toán thành công (SUCCESS).

I. TỔNG HỢP DOANH THU THEO CHI NHÁNH RẠP

Mã CN	Tên Chi nhánh Rạp	Thành phố	Số đơn TT	Số vé hợp lệ	Doanh thu thực thu
CN-HN01	CineViet Vincom Bà Triệu	Hà Nội	36	84	10.140.000 VNĐ
CN-HCM01	CineViet Landmark 81	TP. Hồ Chí Minh	34	79	9.370.000 VNĐ
CN-HP01	CineViet Aeon Mall Hải Phòng	Hải Phòng	29	69	8.325.000 VNĐ
CN-DN01	CineViet Cầu Rồng Đà Nẵng	Đà Nẵng	26	60	7.330.000 VNĐ
TỔNG	**TỔNG CỘNG TOÀN CHUỖI**	-	**125**	**292**	**35.165.000 VNĐ**

II. CHI TIẾT DOANH THU THEO NGÀY VÀ CHI NHÁNH

Ngày TT	Mã CN	Tên Chi nhánh Rạp	Số đơn TT	Số vé hợp lệ	Doanh thu trong ngày
04/11/2026	CN-HCM01	CineViet Landmark 81	4	9	940.000 VNĐ
04/11/2026	CN-HN01	CineViet Vincom Bà Triệu	6	14	1.710.000 VNĐ
04/11/2026	CN-HP01	CineViet Aeon Mall Hải Phòng	4	10	1.350.000 VNĐ
05/11/2026	CN-DN01	CineViet Cầu Rồng Đà Nẵng	6	14	1.710.000 VNĐ
05/11/2026	CN-HCM01	CineViet Landmark 81	6	14	1.710.000 VNĐ
05/11/2026	CN-HN01	CineViet Vincom Bà Triệu	6	14	1.710.000 VNĐ
05/11/2026	CN-HP01	CineViet Aeon Mall Hải Phòng	6	14	1.710.000 VNĐ
06/11/2026	CN-DN01	CineViet Cầu Rồng Đà Nẵng	6	14	1.710.000 VNĐ

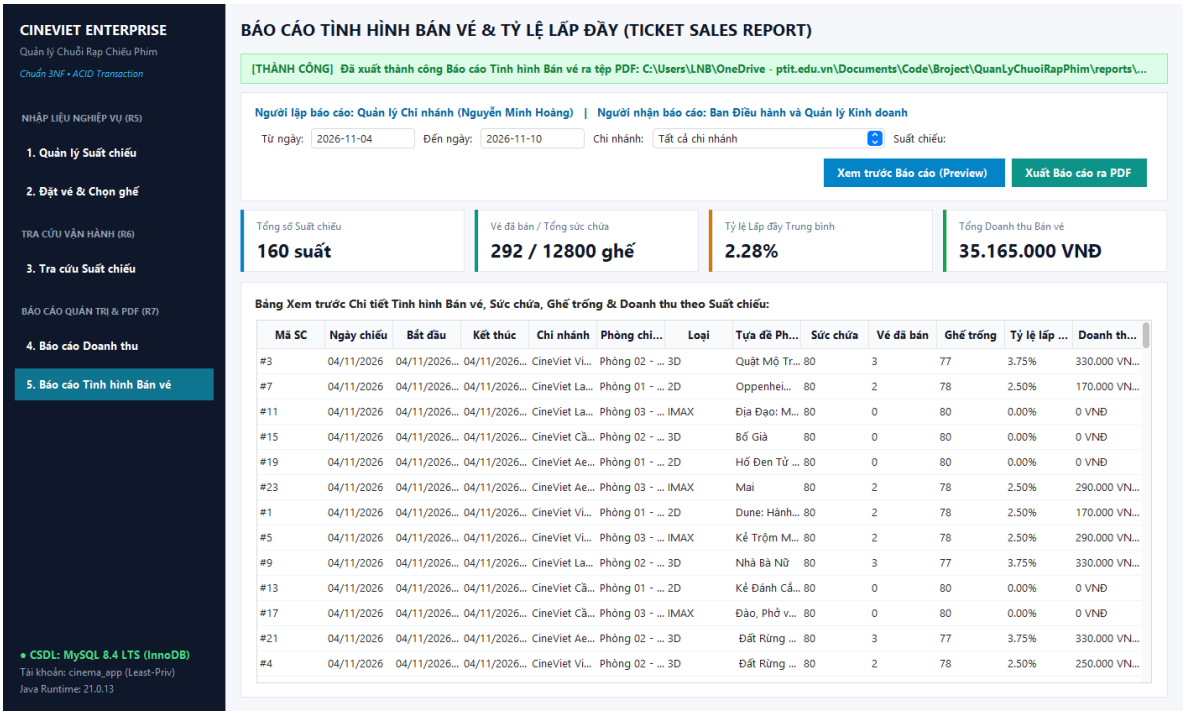
Hình 5.8. Bản kết xuất PDF tiếng Việt Báo cáo Doanh thu Chuỗi rạp (reports/exports/BaoCaoDoanhThu_Mau.pdf).

5.6. Báo cáo Thực tiễn 2: Báo cáo Tình hình Bán vé & Tỷ lệ Lấp đầy — Preview & Xuất PDF (Đáp ứng R7)

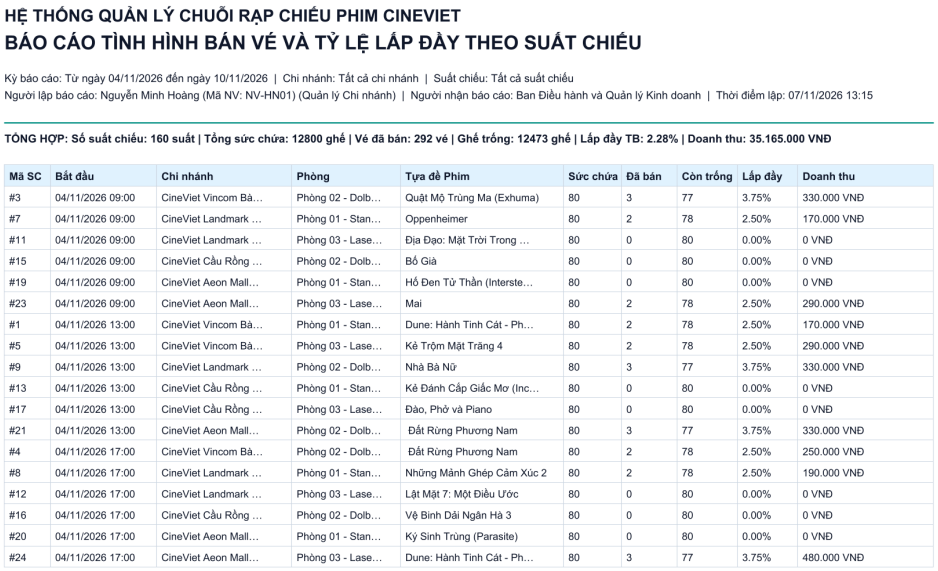
• **Người lập báo cáo:** Quản lý Chi nhánh (`Nguyễn Minh Hoàng`).

• **Người nhận báo cáo:** Ban Điều hành và Quản lý Kinh doanh.

• **Nội dung quản trị:** Thống kê chi tiết từng suất chiếu từ MySQL 8.4 về sức chứa phòng, số vé hợp lệ đã bán, số ghế còn trống, tỷ lệ lấp đầy phòng chiếu (`%`) và doanh thu bán vé thực thu; hỗ trợ xem trước trực quan và kết xuất PDF tiếng Việt.



Hình 5.9. Màn hình Xem trước (Preview) Báo cáo Tình hình Bán vé và Tỷ lệ Lấp đầy theo Suất chiếu.



HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM CINEVIET

BÁO CÁO TÌNH HÌNH BÁN VÉ VÀ TỶ LỆ LẤP ĐẦY THEO SUẤT CHIẾU

Kỳ báo cáo: Từ ngày 04/11/2026 đến ngày 10/11/2026 | Chi nhánh: Tất cả chi nhánh | Suất chiếu: Tất cả suất chiếu

Người lập báo cáo: Nguyễn Minh Hoàng (Mã NV: NV-HN01) (Quản lý Chi nhánh) | Người nhận báo cáo: Ban Điều hành và Quản lý Kinh doanh | Thời điểm lập: 07/11/2026 13:15

TỔNG HỢP: Số suất chiếu: 160 suất | Tổng sức chứa: 12800 ghế | Vé đã bán: 292 vé | Ghế trống: 12473 ghế | Lấp đầy TB: 2.28% | Doanh thu: 35.165.000 VNĐ

Mã SC	Bắt đầu	Chi nhánh	Phòng	Tựa đề Phim	Sức chứa	Đã bán	Còn trống	Lấp đầy	Doanh thu
#3	04/11/2026 09:00	CineViet Vincom Bà…	Phòng 02 - Dolb…	Quật Mộ Trùng Ma (Exhuma)	80	3	77	3.75%	330.000 VNĐ
#7	04/11/2026 09:00	CineViet Landmark …	Phòng 01 - Stan…	Oppenheimer	80	2	78	2.50%	170.000 VNĐ
#11	04/11/2026 09:00	CineViet Landmark …	Phòng 03 - Lase…	Địa Đạo: Mặt Trời Trong …	80	0	80	0.00%	0 VNĐ
#15	04/11/2026 09:00	CineViet Cầu Rồng …	Phòng 02 - Dolb…	Bố Già	80	0	80	0.00%	0 VNĐ
#19	04/11/2026 09:00	CineViet Aeon Mall…	Phòng 01 - Stan…	Hố Đen Tử Thần (Interste…	80	0	80	0.00%	0 VNĐ
#23	04/11/2026 09:00	CineViet Aeon Mall…	Phòng 03 - Lase…	Mai	80	2	78	2.50%	290.000 VNĐ
#1	04/11/2026 13:00	CineViet Vincom Bà…	Phòng 01 - Stan…	Dune: Hành Tinh Cát - Ph…	80	2	78	2.50%	170.000 VNĐ
#5	04/11/2026 13:00	CineViet Vincom Bà…	Phòng 03 - Lase…	Kẻ Trộm Mặt Trăng 4	80	2	78	2.50%	290.000 VNĐ
#9	04/11/2026 13:00	CineViet Landmark …	Phòng 02 - Dolb…	Nhà Bà Nữ	80	3	77	3.75%	330.000 VNĐ
#13	04/11/2026 13:00	CineViet Cầu Rồng …	Phòng 01 - Stan…	Kẻ Đánh Cắp Giấc Mơ (Inc…	80	0	80	0.00%	0 VNĐ
#17	04/11/2026 13:00	CineViet Cầu Rồng …	Phòng 03 - Lase…	Đào, Phở và Piano	80	0	80	0.00%	0 VNĐ
#21	04/11/2026 13:00	CineViet Aeon Mall…	Phòng 02 - Dolb…	Đất Rừng Phương Nam	80	3	77	3.75%	330.000 VNĐ
#4	04/11/2026 17:00	CineViet Vincom Bà…	Phòng 02 - Dolb…	Đất Rừng Phương Nam	80	2	78	2.50%	250.000 VNĐ
#8	04/11/2026 17:00	CineViet Landmark …	Phòng 01 - Stan…	Những Mảnh Ghép Cảm Xúc 2	80	2	78	2.50%	190.000 VNĐ
#12	04/11/2026 17:00	CineViet Landmark …	Phòng 03 - Lase…	Lật Mặt 7: Một Điều Ước	80	0	80	0.00%	0 VNĐ
#16	04/11/2026 17:00	CineViet Cầu Rồng …	Phòng 02 - Dolb…	Vệ Binh Dải Ngân Hà 3	80	0	80	0.00%	0 VNĐ
#20	04/11/2026 17:00	CineViet Aeon Mall…	Phòng 01 - Stan…	Ký Sinh Trùng (Parasite)	80	0	80	0.00%	0 VNĐ
#24	04/11/2026 17:00	CineViet Aeon Mall…	Phòng 03 - Lase…	Dune: Hành Tinh Cát - Ph…	80	3	77	3.75%	480.000 VNĐ

Hình 5.10. Bản kết xuất PDF tiếng Việt Báo cáo Tình hình Bán vé (reports/exports/BaoCaoBanVe_Mau.pdf).

CHƯƠNG 6. TỔNG HỢP KẾT QUẢ KIỂM THỬ TỰ ĐỘNG TRÊN MYSQL COMMUNITY SERVER 8.4.0 LTS

Hệ thống đã được thực thi và kiểm chứng đầu-cuối trên **JDK 21.0.13 LTS (Eclipse Temurin)** và hệ quản trị CSDL chính thức **MySQL Community Server 8.4.0 LTS** (`SELECT VERSION()` trả về `8.4.0`, `@@version_comment` trả về `MySQL Community Server - GPL`, chế độ nghiêm ngặt `ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES`) với quy mô dữ liệu khởi tạo gồm **04 chi nhánh**, **12 phòng chiếu**, **960 ghế vật lý**, **20 bộ phim**, **160 suất chiếu**, **160 đơn đặt vé**, **373 bản ghi vé** (bao gồm 23 ghế hủy được bán lại hợp lệ) và **175 giao dịch thanh toán** (bao gồm 15 đơn có lịch sử `FAILED` + `SUCCESS`).

Bộ Kiểm thử (Test Class)	Cổng Kiểm chứng	Số Test	Kết quả	Nội dung Kiểm chứng Cốt lõi trên MySQL 8.4.0 LTS
`ArchitectureComplianceTest`	Gate B (Kiến trúc)	05	**PASS (5/5)**	Cấm SQL trong UI, tách biệt phân tầng, cấm SQL phá hủy (`DROP`/`TRUNCATE`/`DELETE`) & gọi tiến trình trong `src/main/java`
`TestSafetyGuardTest`	Gate B (Bảo vệ DB)	03	**PASS (3/3)**	Chốt chặn `TestDatabaseGuard` chặn `cinema_chain`, chặn MariaDB, chặn `root` và ẩn mật khẩu trong cấu hình
`ScreeningValidationTest`	Gate A (Đơn vị)	05	**PASS (5/5)**	Kiểm tra hợp lệ thời lượng suất chiếu, giá cơ sở dương, danh sách ghế, thông báo tiếng Việt
`DatabaseIntegrityIT`	Gate C (MySQL 8.4)	13	**PASS (13/13)**	Xác nhận `SELECT VERSION() = 8.4.0`, Trigger chống trùng lịch (`INSERT` & `UPDATE`), chặn đổi phòng khi đã có vé, chặn phòng ngừng hoạt động, chống bán trùng ghế, chống thanh toán đúp, chặn xác nhận thanh toán đơn đã hủy, FK/CHECK/Savepoint, giá lịch sử, chuẩn UTF-8
`ScreeningConcurrencyIT`	Gate C (Đồng thời)	02	**PASS (2/2)**	12 luồng `CREATE` đồng thời và 8 luồng `UPDATE` đồng thời suất chiếu trùng giờ → đúng 01 thắng + chặn đổi phòng
`BookingConcurrencyIT`	Gate C (Đồng thời)	01	**PASS (1/1)**	20 luồng đặt đồng thời cùng 1 ghế → đúng 01 thắng, 0 bản ghi mồ côi; hủy đơn → bán lại ghế; chuyển trạng thái `PENDING`

Bộ Kiểm thử (Test Class)	Cổng Kiểm chứng	Số Test	Kết quả	Nội dung Kiểm chứng Cốt lõi trên MySQL 8.4.0 LTS
`ScreeningSearchIT`	Gate D (Tra cứu)	02	**PASS (2/2)**	Tra cứu đa điều kiện kèm số ghế trống và hiển thị đầy đủ `> 120` suất chiếu `OPEN` trên Màn hình Đặt vé
`ReportAccuracyIT`	Gate E (Báo cáo & PDF)	03	**PASS (3/3)**	Đối chiếu 2 báo cáo & PDF tiếng Việt với SQL độc lập (`35.165.000 VNĐ`, sai lệch `0.00 VNĐ`) và Known-Answer Fixture
TỔNG CỘNG	**Gates A – E**	**34**	**PASS (34/34)**	**0 lỗi biên dịch, 0 lỗi kiểm thử, 0 test bị bỏ qua (Skipped = 0), bảo toàn nguyên vẹn cinema_chain**

PHỤ LỤC A1. MÃ LỆNH DDL KHỞI TẠO 13 BẢNG QUAN HỆ TRÊN MYSQL 8.4 (01_schema.sql)

```
-- ========================================================================
-- TỆP: 01_schema.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 1 - Tạo Bảng] Định nghĩa 13 bảng chuẩn 3NF với đầy đủ
--                PRIMARY KEY, UNIQUE, CHECK, STORED Generated Columns và INDEX.
-- THIẾT KẾ CHUẨN MỰC:
--   - KHÔNG sử dụng `DROP TABLE` hay `SET FOREIGN_KEY_CHECKS = 0` để tránh
--     xóa nhầm dữ liệu khi chạy ở chế độ Normal Setup.
--   - Các ràng buộc Khóa ngoại (16 FOREIGN KEY) được tách riêng sang bước kế tiếp
--     tại tệp `01b_foreign_keys.sql` giúp quy trình khởi tạo tuần tự, rõ ràng.
-- TƯƠNG THÍCH: MySQL Community Server 8.4 LTS (InnoDB, utf8mb4_unicode_ci)
-- ========================================================================

SET NAMES utf8mb4;

-- ------------------------------------------------------------------------
-- 1. Bảng cinema_branch (Chi nhánh rạp chiếu phim)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS cinema_branch (
    branch_id    INT          NOT NULL AUTO_INCREMENT,
    branch_code  VARCHAR(20)  NOT NULL,
    branch_name  VARCHAR(120) NOT NULL,
    address      VARCHAR(255) NOT NULL,
    city         VARCHAR(80)  NOT NULL,
    phone        VARCHAR(20)  NOT NULL,
    active       TINYINT(1)   NOT NULL DEFAULT 1,
    CONSTRAINT pk_cinema_branch  PRIMARY KEY (branch_id),
    CONSTRAINT uq_branch_code    UNIQUE (branch_code),
    CONSTRAINT uq_branch_name    UNIQUE (branch_name),
    CONSTRAINT uq_branch_phone   UNIQUE (phone),
    CONSTRAINT chk_branch_active CHECK (active IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 2. Bảng screening_room (Phòng chiếu phim thuộc chi nhánh)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS screening_room (
    room_id    INT         NOT NULL AUTO_INCREMENT,
    branch_id  INT         NOT NULL,
    room_name  VARCHAR(60) NOT NULL,
    room_type  VARCHAR(20) NOT NULL DEFAULT '2D',
    active     TINYINT(1)  NOT NULL DEFAULT 1,
    CONSTRAINT pk_screening_room   PRIMARY KEY (room_id),
    CONSTRAINT uq_room_branch_name UNIQUE (branch_id, room_name),
    CONSTRAINT chk_room_type       CHECK (room_type IN ('2D', '3D', 'IMAX')),
    CONSTRAINT chk_room_active     CHECK (active IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 3. Bảng seat_type (Danh mục hạng ghế & mức phụ thu chuẩn)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS seat_type (
    seat_type_code VARCHAR(16)   NOT NULL,
    type_name      VARCHAR(60)   NOT NULL,
    surcharge      DECIMAL(12,2) NOT NULL DEFAULT 0.00,

    CONSTRAINT pk_seat_type       PRIMARY KEY (seat_type_code),
    CONSTRAINT uq_seat_type_name  UNIQUE (type_name),
    CONSTRAINT chk_seat_type_code CHECK (seat_type_code IN ('STD', 'VIP', 'COUPLE')),
    CONSTRAINT chk_seat_surcharge CHECK (surcharge >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. Bảng seat (Ghế vật lý cố định trong từng phòng chiếu)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS seat (
    seat_id        INT         NOT NULL AUTO_INCREMENT,
    room_id        INT         NOT NULL,
    seat_row       CHAR(1)     NOT NULL,
    seat_number    INT         NOT NULL,
    seat_type_code VARCHAR(16) NOT NULL DEFAULT 'STD',
    CONSTRAINT pk_seat          PRIMARY KEY (seat_id),
```

```
    CONSTRAINT uq_seat_position UNIQUE (room_id, seat_row, seat_number),
    CONSTRAINT chk_seat_row     CHECK (seat_row >= 'A' AND seat_row <= 'Z'),
    CONSTRAINT chk_seat_number  CHECK (seat_number >= 1 AND seat_number <= 50),
    INDEX idx_seat_type (seat_type_code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. Bảng movie (Phim điện ảnh)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS movie (
    movie_id         INT          NOT NULL AUTO_INCREMENT,
    movie_code       VARCHAR(20)  NOT NULL,
    title            VARCHAR(180) NOT NULL,
    duration_minutes INT          NOT NULL,
    release_date     DATE         NOT NULL,
    age_rating       VARCHAR(10)  NOT NULL DEFAULT 'P',
    country          VARCHAR(80)  NOT NULL,
    description      TEXT         NULL,
    CONSTRAINT pk_movie         PRIMARY KEY (movie_id),
    CONSTRAINT uq_movie_code    UNIQUE (movie_code),
    CONSTRAINT chk_movie_dur    CHECK (duration_minutes >= 1 AND duration_minutes <= 400),
    CONSTRAINT chk_movie_rating CHECK (age_rating IN ('P', 'K', 'T13', 'T16', 'T18'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 6. Bảng genre (Thể loại phim)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS genre (
    genre_id    INT          NOT NULL AUTO_INCREMENT,
    genre_name  VARCHAR(80)  NOT NULL,
    description VARCHAR(255) NULL,
    CONSTRAINT pk_genre      PRIMARY KEY (genre_id),
    CONSTRAINT uq_genre_name UNIQUE (genre_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. Bảng movie_genre (Liên kết N-N giữa Phim và Thể loại)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS movie_genre (

    movie_id INT NOT NULL,
    genre_id INT NOT NULL,
    CONSTRAINT pk_movie_genre PRIMARY KEY (movie_id, genre_id),
    INDEX idx_mg_genre (genre_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 8. Bảng screening (Suất chiếu)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS screening (
    screening_id INT           NOT NULL AUTO_INCREMENT,
    movie_id     INT           NOT NULL,
    room_id      INT           NOT NULL,
    start_time   DATETIME      NOT NULL,
    end_time     DATETIME      NOT NULL,
    base_price   DECIMAL(12,2) NOT NULL,
    status       VARCHAR(20)   NOT NULL DEFAULT 'OPEN',
    CONSTRAINT pk_screening        PRIMARY KEY (screening_id),
    CONSTRAINT chk_screening_time  CHECK (end_time > start_time),
    CONSTRAINT chk_screening_price CHECK (base_price > 0),
    CONSTRAINT chk_screening_stat  CHECK (status IN ('OPEN', 'CANCELLED')),
    INDEX idx_screening_room_time (room_id, start_time, end_time),
    INDEX idx_screening_movie_time (movie_id, start_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 9. Bảng customer (Khách hàng thành viên)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS customer (
    customer_id INT          NOT NULL AUTO_INCREMENT,
    full_name   VARCHAR(120) NOT NULL,
    phone       VARCHAR(20)  NOT NULL,
    email       VARCHAR(120) NOT NULL,
    birth_date  DATE         NULL,
    created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_customer       PRIMARY KEY (customer_id),
```

```
    CONSTRAINT uq_customer_phone UNIQUE (phone),
    CONSTRAINT uq_customer_email UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 10. Bảng employee (Nhân viên làm việc tại chi nhánh)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS employee (
    employee_id   INT          NOT NULL AUTO_INCREMENT,
    branch_id     INT          NOT NULL,
    employee_code VARCHAR(20)  NOT NULL,
    full_name     VARCHAR(120) NOT NULL,
    phone         VARCHAR(20)  NOT NULL,
    role          VARCHAR(20)  NOT NULL DEFAULT 'CASHIER',
    hired_date    DATE         NOT NULL,
    CONSTRAINT pk_employee      PRIMARY KEY (employee_id),
    CONSTRAINT uq_employee_code UNIQUE (employee_code),
    CONSTRAINT uq_employee_phone UNIQUE (phone),
    CONSTRAINT chk_employee_role CHECK (role IN ('CASHIER', 'MANAGER', 'ACCOUNTANT')),

    INDEX idx_employee_branch (branch_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 11. Bảng booking (Đơn đặt vé; customer_id NULLable cho Khách vãng lai)
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS booking (
    booking_id     INT         NOT NULL AUTO_INCREMENT,
    booking_code   VARCHAR(30) NOT NULL,
    screening_id   INT         NOT NULL,
    customer_id    INT         NULL,
    employee_id    INT         NOT NULL,
    booked_at      DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    booking_status VARCHAR(20) NOT NULL DEFAULT 'CONFIRMED',
    CONSTRAINT pk_booking              PRIMARY KEY (booking_id),
    CONSTRAINT uq_booking_code         UNIQUE (booking_code),
    CONSTRAINT uq_booking_id_screening UNIQUE (booking_id, screening_id),
    CONSTRAINT chk_booking_status      CHECK (booking_status IN ('CONFIRMED', 'PENDING', 'CANCEL
LED')),
    INDEX idx_booking_screening (screening_id, booking_status),
    INDEX idx_booking_customer (customer_id),
    INDEX idx_booking_employee (employee_id),
    INDEX idx_booking_booked_at (booked_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 12. Bảng ticket (Vé xem phim chi tiết theo ghế)
--     active_seat_key đảm bảo không thể có 2 vé VALID cùng (screening_id, seat_id),
--     đồng thời giải phóng ghế (về NULL) khi vé chuyển sang CANCELLED.
-- ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS ticket (
    ticket_id       INT           NOT NULL AUTO_INCREMENT,
    booking_id      INT           NOT NULL,
    screening_id    INT           NOT NULL,
    seat_id         INT           NOT NULL,
    unit_price      DECIMAL(12,2) NOT NULL,
    ticket_status   VARCHAR(20)   NOT NULL DEFAULT 'VALID',
    cancelled_at    DATETIME      NULL,
    active_seat_key VARCHAR(64)   GENERATED ALWAYS AS (
        CASE WHEN ticket_status = 'VALID' THEN CONCAT(screening_id, '-', seat_id) ELSE NULL END
    ) STORED,
    CONSTRAINT pk_ticket             PRIMARY KEY (ticket_id),
    CONSTRAINT uq_ticket_active_seat UNIQUE (active_seat_key),
    CONSTRAINT chk_ticket_price      CHECK (unit_price >= 0),
    CONSTRAINT chk_ticket_status     CHECK (ticket_status IN ('VALID', 'CANCELLED')),
    INDEX idx_ticket_screening_status (screening_id, ticket_status),
    INDEX idx_ticket_booking_status (booking_id, ticket_status),
    INDEX idx_ticket_booking_scr (booking_id, screening_id),
    INDEX idx_ticket_seat (seat_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 13. Bảng payment (Giao dịch thanh toán)
--     active_success_booking đảm bảo mỗi đơn đặt vé chỉ có tối đa 01 giao dịch
--     ở trạng thái SUCCESS, chống thanh toán đúp và nhân bản doanh thu.

-- ------------------------------------------------------------------------
```

```
CREATE TABLE IF NOT EXISTS payment (
    payment_id             INT           NOT NULL AUTO_INCREMENT,
    booking_id             INT           NOT NULL,
    amount                 DECIMAL(12,2) NOT NULL,
    payment_method         VARCHAR(20)   NOT NULL DEFAULT 'CASH',
    payment_status         VARCHAR(20)   NOT NULL DEFAULT 'SUCCESS',
    paid_at                DATETIME      NULL,
    transaction_ref        VARCHAR(60)   NULL,
    active_success_booking INT           GENERATED ALWAYS AS (
        CASE WHEN payment_status = 'SUCCESS' THEN booking_id ELSE NULL END
    ) STORED,
    CONSTRAINT pk_payment             PRIMARY KEY (payment_id),
    CONSTRAINT uq_payment_one_success UNIQUE (active_success_booking),
    CONSTRAINT chk_payment_amount     CHECK (amount >= 0),
    CONSTRAINT chk_payment_method     CHECK (payment_method IN ('CASH', 'CARD', 'TRANSFER', 'EWA
LLET')),
    CONSTRAINT chk_payment_status     CHECK (payment_status IN ('PENDING', 'SUCCESS', 'FAILED',
'REFUNDED')),
    INDEX idx_payment_status_paid_at (payment_status, paid_at),
    INDEX idx_payment_booking (booking_id, payment_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

PHỤ LỤC A2. MÃ LỆNH THIẾT LẬP 16 RÀNG BUỘC KHÓA NGOẠI (01b_foreign_keys.sql)

```
-- ========================================================================
-- TỆP: 01b_foreign_keys.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 2 - Tạo Khóa ngoại] Thiết lập toàn bộ 16 ràng buộc
--                tham chiếu FOREIGN KEY giữa 13 bảng thực thể sau khi tạo bảng.
-- LƯU Ý TƯƠNG THÍCH MYSQL 8.4 LTS:
--   - Các bảng `ticket` và `payment` có sử dụng cột sinh vật lý (`GENERATED ALWAYS
--     AS ... STORED`). Theo quy định của MySQL 8.4 InnoDB (mã lỗi 3108), các cột
--     cơ sở tham gia biểu thức STORED GENERATED không được dùng `ON UPDATE CASCADE`,
--     do đó được thiết kế chuẩn xác với `ON DELETE RESTRICT ON UPDATE RESTRICT`.
-- ========================================================================

SET NAMES utf8mb4;

-- 1. Phòng chiếu thuộc Chi nhánh rạp (1 - N)
ALTER TABLE screening_room
    ADD CONSTRAINT fk_room_branch FOREIGN KEY (branch_id)
        REFERENCES cinema_branch (branch_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 2. Ghế vật lý thuộc Phòng chiếu & phân loại theo Hạng ghế
ALTER TABLE seat
    ADD CONSTRAINT fk_seat_room FOREIGN KEY (room_id)
        REFERENCES screening_room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_seat_type FOREIGN KEY (seat_type_code)
        REFERENCES seat_type (seat_type_code)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 3. Bảng trung gian Phim - Thể loại (N - N)
ALTER TABLE movie_genre
    ADD CONSTRAINT fk_mg_movie FOREIGN KEY (movie_id)
        REFERENCES movie (movie_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    ADD CONSTRAINT fk_mg_genre FOREIGN KEY (genre_id)
        REFERENCES genre (genre_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 4. Suất chiếu gắn với Phim và Phòng chiếu
ALTER TABLE screening
    ADD CONSTRAINT fk_screening_movie FOREIGN KEY (movie_id)
        REFERENCES movie (movie_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_screening_room FOREIGN KEY (room_id)
        REFERENCES screening_room (room_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 5. Nhân viên làm việc tại Chi nhánh rạp
ALTER TABLE employee
    ADD CONSTRAINT fk_employee_branch FOREIGN KEY (branch_id)
        REFERENCES cinema_branch (branch_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 6. Đơn đặt vé gắn với Suất chiếu, Khách hàng (NULLable cho khách vãng lai) & Nhân viên
ALTER TABLE booking

    ADD CONSTRAINT fk_booking_screening FOREIGN KEY (screening_id)
        REFERENCES screening (screening_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    ADD CONSTRAINT fk_booking_customer FOREIGN KEY (customer_id)
        REFERENCES customer (customer_id)
        ON DELETE SET NULL ON UPDATE CASCADE,
    ADD CONSTRAINT fk_booking_employee FOREIGN KEY (employee_id)
        REFERENCES employee (employee_id)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- 7. Vé xem phim gắn với Đơn đặt vé, Cặp (booking_id, screening_id), Suất chiếu & Ghế
ALTER TABLE ticket
    ADD CONSTRAINT fk_ticket_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    ADD CONSTRAINT fk_ticket_booking_scr FOREIGN KEY (booking_id, screening_id)
```

```
        REFERENCES booking (booking_id, screening_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    ADD CONSTRAINT fk_ticket_screening FOREIGN KEY (screening_id)
        REFERENCES screening (screening_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT,
    ADD CONSTRAINT fk_ticket_seat FOREIGN KEY (seat_id)
        REFERENCES seat (seat_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT;

-- 8. Giao dịch thanh toán gắn với Đơn đặt vé
ALTER TABLE payment
    ADD CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id)
        REFERENCES booking (booking_id)
        ON DELETE RESTRICT ON UPDATE RESTRICT;
```

PHỤ LỤC B. MÃ LỆNH TRIGGER BẢO VỆ TOÀN VẸN NGHIỆP VỤ (02_triggers.sql)

```
-- ========================================================================
-- TỆP: 02_triggers.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 3 - Tạo Trigger] Thiết lập 06 Trigger bảo vệ quy tắc
--                nghiệp vụ trực tiếp ở tầng Hệ quản trị MySQL 8.4 LTS:
--   1. Chống trùng lịch chiếu trong cùng phòng & đảm bảo thời lượng tối thiểu.
--   2. Đảm bảo ghế trên vé thuộc đúng phòng chiếu của suất chiếu & đúng đơn.
--   3. Đảm bảo tính nhất quán của thời điểm hủy vé và thời điểm thanh toán.
-- LƯU Ý KỸ THUẬT MYSQL 8.4:
--   MySQL 8.4 không hỗ trợ cú pháp `CREATE OR REPLACE TRIGGER` (khác với View),
--   do đó sử dụng `DROP TRIGGER IF EXISTS` cho siêu dữ liệu thủ tục này để đảm bảo
--   tính lũy đẳng (idempotent) khi chạy Normal Setup mà hoàn toàn không ảnh hưởng
--   tới bất kỳ dữ liệu nào trong các bảng.
-- ========================================================================

SET NAMES utf8mb4;

DROP TRIGGER IF EXISTS trg_screening_validate_ins;
DROP TRIGGER IF EXISTS trg_screening_validate_upd;
DROP TRIGGER IF EXISTS trg_ticket_validate_ins;
DROP TRIGGER IF EXISTS trg_ticket_validate_upd;
DROP TRIGGER IF EXISTS trg_payment_validate_ins;
DROP TRIGGER IF EXISTS trg_payment_validate_upd;

DELIMITER $$

-- ------------------------------------------------------------------------
-- 1. Trigger kiểm tra khi THÊM MỚI suất chiếu (screening)
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_screening_validate_ins
BEFORE INSERT ON screening
FOR EACH ROW
BEGIN
    DECLARE v_duration INT DEFAULT 0;
    DECLARE v_room_active TINYINT DEFAULT 0;
    DECLARE v_overlap_count INT DEFAULT 0;

    IF NEW.end_time <= NEW.start_time THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Thoi gian ket thuc phai lon hon thoi gian
 bat dau.';
    END IF;

    IF NEW.base_price <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-04]: Gia ve co so cua suat chieu phai lon hon
0.';
    END IF;

    SELECT duration_minutes INTO v_duration
    FROM movie
    WHERE movie_id = NEW.movie_id;

    IF TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) < v_duration THEN
        SIGNAL SQLSTATE '45000'

            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Khoang thoi gian suat chieu nho hon thoi
luong phim.';
    END IF;

    SELECT active INTO v_room_active
    FROM screening_room
    WHERE room_id = NEW.room_id;

    IF v_room_active = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-01]: Phong chieu dang tam ngung hoat dong.';
    END IF;

    IF NEW.status = 'OPEN' THEN
        SELECT COUNT(*) INTO v_overlap_count
        FROM screening s
```

```
        WHERE s.room_id = NEW.room_id
          AND s.status = 'OPEN'
          AND s.start_time < NEW.end_time
          AND s.end_time > NEW.start_time;

        IF v_overlap_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-06]: Trung lich chieu voi mot suat chieu k
hac trong cung phong chieu.';
        END IF;
    END IF;
END$$

-- ------------------------------------------------------------------------
-- 2. Trigger kiểm tra khi CẬP NHẬT suất chiếu (screening)
-- ------------------------------------------------------------------------
CREATE TRIGGER trg_screening_validate_upd
BEFORE UPDATE ON screening
FOR EACH ROW
BEGIN
    DECLARE v_duration INT DEFAULT 0;
    DECLARE v_room_active TINYINT DEFAULT 0;
    DECLARE v_overlap_count INT DEFAULT 0;
    DECLARE v_ticket_count INT DEFAULT 0;
    DECLARE v_valid_ticket_count INT DEFAULT 0;

    IF NEW.end_time <= NEW.start_time THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Thoi gian ket thuc phai lon hon thoi gian
 bat dau.';
    END IF;

    IF NEW.base_price <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-04]: Gia ve co so cua suat chieu phai lon hon
0.';
    END IF;

    SELECT active INTO v_room_active
    FROM screening_room
    WHERE room_id = NEW.room_id;

    IF v_room_active = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-01]: Phong chieu dang tam ngung hoat dong.';
    END IF;

    IF NEW.room_id <> OLD.room_id THEN
        SELECT COUNT(*) INTO v_ticket_count
        FROM ticket
        WHERE screening_id = OLD.screening_id;

        IF v_ticket_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Khong the doi phong chieu cho suat ch
ieu da phat sinh ve.';
        END IF;
    END IF;

    IF OLD.status = 'OPEN' AND NEW.status = 'CANCELLED' THEN
        SELECT COUNT(*) INTO v_valid_ticket_count
        FROM ticket
        WHERE screening_id = OLD.screening_id
          AND ticket_status = 'VALID';

        IF v_valid_ticket_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the huy suat chieu dang co ve h
op le chua duoc hoan huy.';
        END IF;
    END IF;

    SELECT duration_minutes INTO v_duration
    FROM movie
    WHERE movie_id = NEW.movie_id;
```

```
    IF TIMESTAMPDIFF(MINUTE, NEW.start_time, NEW.end_time) < v_duration THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-05]: Khoang thoi gian suat chieu nho hon thoi
luong phim.';
    END IF;

    IF NEW.status = 'OPEN' THEN
        SELECT COUNT(*) INTO v_overlap_count
        FROM screening s
        WHERE s.room_id = NEW.room_id
          AND s.screening_id <> NEW.screening_id
          AND s.status = 'OPEN'
          AND s.start_time < NEW.end_time
          AND s.end_time > NEW.start_time;

        IF v_overlap_count > 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Loi nghiep vu [BR-06]: Trung lich chieu voi mot suat chieu k

hac trong cung phong chieu.';
        END IF;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 3. Trigger kiểm tra khi THÊM MỚI vé xem phim (ticket)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_ticket_validate_ins
BEFORE INSERT ON ticket
FOR EACH ROW
BEGIN
    DECLARE v_scr_room_id INT DEFAULT NULL;
    DECLARE v_scr_status VARCHAR(20) DEFAULT NULL;
    DECLARE v_seat_room_id INT DEFAULT NULL;
    DECLARE v_booking_scr_id INT DEFAULT NULL;

    SELECT room_id, status INTO v_scr_room_id, v_scr_status
    FROM screening
    WHERE screening_id = NEW.screening_id;

    SELECT room_id INTO v_seat_room_id
    FROM seat
    WHERE seat_id = NEW.seat_id;

    SELECT screening_id INTO v_booking_scr_id
    FROM booking
    WHERE booking_id = NEW.booking_id;

    IF v_booking_scr_id IS NULL OR v_booking_scr_id <> NEW.screening_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Suat chieu tren ve khong khop voi suat ch
ieu cua don dat ve.';
    END IF;

    IF v_scr_room_id IS NULL OR v_seat_room_id IS NULL OR v_scr_room_id <> v_seat_room_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Ghe duoc chon khong thuoc phong chieu cua
 suat chieu nay.';
    END IF;

    IF NEW.ticket_status = 'VALID' AND v_scr_status <> 'OPEN' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-08]: Khong the phat hanh ve hop le cho suat ch
ieu da bi huy.';
    END IF;

    IF NEW.ticket_status = 'CANCELLED' AND NEW.cancelled_at IS NULL THEN
        SET NEW.cancelled_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 4. Trigger kiểm tra khi CẬP NHẬT vé xem phim (ticket)
-- ----------------------------------------------------------------------------

CREATE TRIGGER trg_ticket_validate_upd
BEFORE UPDATE ON ticket
```

```
FOR EACH ROW
BEGIN
    DECLARE v_scr_room_id INT DEFAULT NULL;
    DECLARE v_seat_room_id INT DEFAULT NULL;

    SELECT room_id INTO v_scr_room_id
    FROM screening
    WHERE screening_id = NEW.screening_id;

    SELECT room_id INTO v_seat_room_id
    FROM seat
    WHERE seat_id = NEW.seat_id;

    IF v_scr_room_id <> v_seat_room_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-09]: Ghe duoc chon khong thuoc phong chieu cua
 suat chieu nay.';
    END IF;

    IF NEW.ticket_status = 'CANCELLED' AND NEW.cancelled_at IS NULL THEN
        SET NEW.cancelled_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 5. Trigger kiểm tra khi THÊM MỚI giao dịch thanh toán (payment)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_payment_validate_ins
BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-13]: So tien thanh toan phai lon hon 0.';
    END IF;

    IF NEW.payment_status = 'SUCCESS' AND NEW.paid_at IS NULL THEN
        SET NEW.paid_at = CURRENT_TIMESTAMP;
    END IF;
END$$

-- ----------------------------------------------------------------------------
-- 6. Trigger kiểm tra khi CẬP NHẬT giao dịch thanh toán (payment)
-- ----------------------------------------------------------------------------
CREATE TRIGGER trg_payment_validate_upd
BEFORE UPDATE ON payment
FOR EACH ROW
BEGIN
    IF NEW.amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Loi nghiep vu [BR-13]: So tien thanh toan phai lon hon 0.';
    END IF;

    IF NEW.payment_status = 'SUCCESS' AND NEW.paid_at IS NULL THEN
        SET NEW.paid_at = CURRENT_TIMESTAMP;
    END IF;
END$$

DELIMITER ;
```

PHỤ LỤC C. MÃ LỆNH VIEW TRA CỨU & ĐỐI SOÁT DOANH THU (03_views.sql)

```
-- ============================================================================
-- TỆP: 03_views.sql
-- DỰ ÁN: HỆ THỐNG QUẢN LÝ CHUỖI RẠP CHIẾU PHIM
-- BƯỚC KHỞI TẠO: [Bước 4 - Tạo View] Định nghĩa 04 khung nhìn (Views) phục vụ
--                Tra cứu Suất chiếu, Báo cáo Doanh thu và Báo cáo Bán vé theo
--                đúng quy tắc ghi nhận doanh thu (BR-14 & BR-15: chống nhân bản).
-- TƯƠNG THÍCH MYSQL 8.4 LTS:
--   - Sử dụng `CREATE OR REPLACE VIEW` chuẩn của MySQL 8.4 (không cần `DROP VIEW`),
--     tuân thủ nghiêm ngặt chế độ `ONLY_FULL_GROUP_BY`.
-- ============================================================================

SET NAMES utf8mb4;

-- ----------------------------------------------------------------------------
-- 1. Khung nhìn vw_screening_search (Tra cứu suất chiếu & ghế trống)
-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_screening_search AS
SELECT
    s.screening_id,
    s.movie_id,
    m.movie_code,
    m.title AS movie_title,
    m.duration_minutes,
    m.age_rating,
    s.room_id,
    r.room_name,
    r.room_type,
    r.branch_id,
    b.branch_code,
    b.branch_name,
    b.city,
    s.start_time,
    s.end_time,
    DATE(s.start_time) AS screening_date,
    s.base_price,
    s.status AS screening_status,
    (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) AS total_seats,
    (SELECT COUNT(*) FROM ticket tk WHERE tk.screening_id = s.screening_id AND tk.ticket_status
= 'VALID') AS sold_seats,
    (
        (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) -
        (SELECT COUNT(*) FROM ticket tk WHERE tk.screening_id = s.screening_id AND tk.ticket_sta
tus = 'VALID')
    ) AS available_seats
FROM screening s
JOIN movie m          ON m.movie_id = s.movie_id
JOIN screening_room r ON r.room_id = s.room_id
JOIN cinema_branch b  ON b.branch_id = r.branch_id;

-- ----------------------------------------------------------------------------
-- 2. Khung nhìn vw_verified_booking_revenue (Đơn đặt vé đủ điều kiện doanh thu)
--    Quy tắc BR-14 & BR-15:
--    - Chỉ tính đơn có booking_status = 'CONFIRMED'
--    - Có đúng 1 giao dịch payment_status = 'SUCCESS'
--    - Chỉ cộng các vé có ticket_status = 'VALID'

-- ----------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_verified_booking_revenue AS
SELECT
    bk.booking_id,
    bk.booking_code,
    bk.screening_id,
    sc.movie_id,
    mv.title AS movie_title,
    sc.room_id,
    rm.room_name,
    rm.branch_id,
    br.branch_code,
    br.branch_name,
    bk.customer_id,
    COALESCE(cu.full_name, 'Khách vãng lai') AS customer_name,
    bk.employee_id,
```

```
    em.full_name AS employee_name,
    bk.booked_at,
    pm.payment_id,
    pm.payment_method,
    pm.paid_at,
    DATE(pm.paid_at) AS revenue_date,
    COUNT(tk.ticket_id) AS valid_ticket_count,
    COALESCE(SUM(tk.unit_price), 0.00) AS recognized_revenue
FROM booking bk
JOIN screening sc      ON sc.screening_id = bk.screening_id
JOIN movie mv          ON mv.movie_id = sc.movie_id
JOIN screening_room rm ON rm.room_id = sc.room_id
JOIN cinema_branch br  ON br.branch_id = rm.branch_id
LEFT JOIN customer cu  ON cu.customer_id = bk.customer_id
JOIN employee em       ON em.employee_id = bk.employee_id
JOIN payment pm        ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
JOIN ticket tk         ON tk.booking_id = bk.booking_id AND tk.ticket_status = 'VALID'
WHERE bk.booking_status = 'CONFIRMED'
GROUP BY
    bk.booking_id,
    bk.booking_code,
    bk.screening_id,
    sc.movie_id,
    mv.title,
    sc.room_id,
    rm.room_name,
    rm.branch_id,
    br.branch_code,
    br.branch_name,
    bk.customer_id,
    cu.full_name,
    bk.employee_id,
    em.full_name,
    bk.booked_at,
    pm.payment_id,
    pm.payment_method,
    pm.paid_at;

-- ------------------------------------------------------------------------

-- 3. Khung nhìn vw_branch_daily_revenue (Doanh thu theo ngày & chi nhánh)
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_branch_daily_revenue AS
SELECT
    revenue_date,
    branch_id,
    branch_code,
    branch_name,
    COUNT(DISTINCT booking_id) AS paid_booking_count,
    SUM(valid_ticket_count)    AS paid_ticket_count,
    SUM(recognized_revenue)    AS total_paid_revenue
FROM vw_verified_booking_revenue
GROUP BY
    revenue_date,
    branch_id,
    branch_code,
    branch_name;

-- ------------------------------------------------------------------------
-- 4. Khung nhìn vw_screening_occupancy (Báo cáo bán vé & tỷ lệ lấp đầy suất chiếu)
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW vw_screening_occupancy AS
SELECT
    s.screening_id,
    DATE(s.start_time) AS screening_date,
    s.start_time,
    s.end_time,
    s.status AS screening_status,
    s.base_price,
    m.movie_id,
    m.movie_code,
    m.title AS movie_title,
    r.room_id,
    r.room_name,
    r.room_type,
    b.branch_id,
```

```
    b.branch_code,
    b.branch_name,
    (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) AS room_capacity,
    (
        SELECT COUNT(*)
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
        WHERE tk.screening_id = s.screening_id
          AND tk.ticket_status = 'VALID'
          AND bk.booking_status = 'CONFIRMED'
    ) AS tickets_sold,
    (
        (SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id) -
        (SELECT COUNT(*) FROM ticket tk WHERE tk.screening_id = s.screening_id AND tk.ticket_sta
tus = 'VALID')
    ) AS available_seats,
    ROUND(
        100.0 * (
            SELECT COUNT(*)
            FROM ticket tk
            JOIN booking bk ON bk.booking_id = tk.booking_id
            JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
            WHERE tk.screening_id = s.screening_id
              AND tk.ticket_status = 'VALID'
              AND bk.booking_status = 'CONFIRMED'
        ) / NULLIF((SELECT COUNT(*) FROM seat st WHERE st.room_id = s.room_id), 0),
        2
    ) AS occupancy_rate_pct,
    COALESCE((
        SELECT SUM(tk.unit_price)
        FROM ticket tk
        JOIN booking bk ON bk.booking_id = tk.booking_id
        JOIN payment pm ON pm.booking_id = bk.booking_id AND pm.payment_status = 'SUCCESS'
        WHERE tk.screening_id = s.screening_id
          AND tk.ticket_status = 'VALID'
          AND bk.booking_status = 'CONFIRMED'
    ), 0.00) AS sales_revenue
FROM screening s
JOIN movie m          ON m.movie_id = s.movie_id
JOIN screening_room r ON r.room_id = s.room_id
JOIN cinema_branch b  ON b.branch_id = r.branch_id;
```

PHỤ LỤC D. CÁC TRUY VẤN NGHIỆP VỤ, GIAO DỊCH & BÁO CÁO (business_queries.sql)

```
-- ============================================================================
-- TỆP: business_queries.sql
-- MỤC ĐÍCH: Bộ câu lệnh SQL chuẩn phục vụ 5 màn hình/báo cáo nghiệp vụ (R5, R6, R7)
--           và Phụ lục SQL trong Báo cáo học thuật (R8).
-- ============================================================================

SET NAMES utf8mb4;

-- ----------------------------------------------------------------------------
-- Q1. Kiểm tra xung đột lịch chiếu trong cùng một phòng chiếu (Screen 1 - R5)
-- ----------------------------------------------------------------------------
SELECT
    s.screening_id,
    m.title AS movie_title,
    s.start_time,
    s.end_time,
    s.status
FROM screening s
JOIN movie m ON m.movie_id = s.movie_id
WHERE s.room_id = 1
  AND s.status = 'OPEN'
  AND s.start_time < '2026-11-04 11:30:00'
  AND s.end_time > '2026-11-04 09:00:00';

-- ----------------------------------------------------------------------------
-- Q2. Truy vấn sơ đồ ghế và trạng thái ghế theo suất chiếu (Screen 2 - R5)
-- ----------------------------------------------------------------------------
SELECT
    st.seat_id,
    st.seat_row,
    st.seat_number,
    CONCAT(st.seat_row, LPAD(st.seat_number, 2, '0')) AS seat_label,
    st.seat_type_code,
    tp.type_name,
    tp.surcharge,
    (sc.base_price + tp.surcharge) AS final_ticket_price,
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM ticket tk
            WHERE tk.screening_id = sc.screening_id
              AND tk.seat_id = st.seat_id
              AND tk.ticket_status = 'VALID'
        ) THEN 'OCCUPIED'
        ELSE 'AVAILABLE'
    END AS seat_availability
FROM screening sc
JOIN seat st      ON st.room_id = sc.room_id
JOIN seat_type tp ON tp.seat_type_code = st.seat_type_code
WHERE sc.screening_id = 1
ORDER BY st.seat_row, st.seat_number;

-- ----------------------------------------------------------------------------
-- Q3. Tra cứu suất chiếu theo tên phim, chi nhánh, ngày chiếu, khung giờ
--     kèm số lượng ghế trống chính xác (Screen 3 - R6)

-- ----------------------------------------------------------------------------
SELECT
    v.screening_id,
    v.movie_code,
    v.movie_title,
    v.duration_minutes,
    v.age_rating,
    v.branch_name,
    v.room_name,
    v.room_type,
    v.start_time,
    v.end_time,
    v.base_price,
    v.total_seats,
    v.sold_seats,
    v.available_seats,
```

```
    v.screening_status
FROM vw_screening_search v
WHERE v.screening_status = 'OPEN'
  AND v.screening_date = '2026-11-04'
ORDER BY v.start_time ASC, v.branch_name ASC
LIMIT 15;

-- ------------------------------------------------------------------------
-- Q4. Báo cáo Doanh thu theo Chi nhánh và Khoảng thời gian thanh toán (Screen 4 - R7)
--     Người lập: Kế toán | Người nhận: Giám đốc kinh doanh chuỗi rạp
--     Chỉ tính đơn CONFIRMED có thanh toán SUCCESS và vé VALID (chống nhân bản)
-- ------------------------------------------------------------------------
SELECT
    br.branch_id,
    br.branch_code,
    br.branch_name,
    br.city,
    COUNT(DISTINCT vr.booking_id)         AS total_paid_bookings,
    COALESCE(SUM(vr.valid_ticket_count), 0) AS total_valid_tickets,
    COALESCE(SUM(vr.recognized_revenue), 0.00) AS total_paid_revenue
FROM cinema_branch br
LEFT JOIN vw_verified_booking_revenue vr
    ON vr.branch_id = br.branch_id
   AND vr.revenue_date BETWEEN '2026-11-01' AND '2026-11-15'
GROUP BY br.branch_id, br.branch_code, br.branch_name, br.city
ORDER BY total_paid_revenue DESC, br.branch_id ASC;

-- ------------------------------------------------------------------------
-- Q5. Chi tiết doanh thu theo ngày và chi nhánh (Screen 4 Detail Table - R7)
-- ------------------------------------------------------------------------
SELECT
    revenue_date,
    branch_code,
    branch_name,
    paid_booking_count,
    paid_ticket_count,
    total_paid_revenue
FROM vw_branch_daily_revenue
WHERE revenue_date BETWEEN '2026-11-01' AND '2026-11-15'

ORDER BY revenue_date ASC, branch_code ASC;

-- ------------------------------------------------------------------------
-- Q6. Báo cáo Tình hình Bán vé & Tỷ lệ Lấp đầy theo Suất chiếu (Screen 5 - R7)
--     Người lập: Quản lý chi nhánh | Người nhận: Ban điều hành & Quản lý kinh doanh
-- ------------------------------------------------------------------------
SELECT
    screening_id,
    screening_date,
    TIME(start_time) AS start_clock,
    TIME(end_time)   AS end_clock,
    branch_name,
    room_name,
    room_type,
    movie_title,
    room_capacity,
    tickets_sold,
    available_seats,
    occupancy_rate_pct,
    sales_revenue
FROM vw_screening_occupancy
WHERE screening_date BETWEEN '2026-11-04' AND '2026-11-10'
ORDER BY screening_date ASC, start_time ASC, branch_id ASC
LIMIT 20;

-- ------------------------------------------------------------------------
-- Q7. Đối soát toàn vẹn Doanh thu Độc lập (Gate E Verification Query):
--     Chứng minh tổng doanh thu từ bảng ticket (thuộc đơn có payment SUCCESS)
--     bằng chính xác tổng doanh thu trên View báo cáo, không bị nhân bản bởi
--     các đơn có nhiều dòng payment (FAILED + SUCCESS).
-- ------------------------------------------------------------------------
SELECT
    (SELECT SUM(tk.unit_price)
     FROM ticket tk
     JOIN booking bk ON bk.booking_id = tk.booking_id
     WHERE tk.ticket_status = 'VALID'
```

```
   AND bk.booking_status = 'CONFIRMED'
   AND EXISTS (
       SELECT 1 FROM payment pm
       WHERE pm.booking_id = bk.booking_id
         AND pm.payment_status = 'SUCCESS'
   )
) AS independent_ticket_revenue_sum,
(SELECT SUM(total_paid_revenue)
 FROM vw_branch_daily_revenue
) AS view_report_revenue_sum;
```